TU Clausthal

MASTER THESIS

An Intelligent Analysis Approach of Real Test Drive Recordings Based on Self-Supervised Learning and HIL Real-Time Simulation Dataset

Yahia Amir Yahia Gamal

Faculty of Mathematics/Computer Science and Mechanical Engineering
Institute of Computer Science
Institute for Software and Systems Engineering

Matriculation Number	539500
Study Programme	MSc Computer Science
First Examiner	Dr. Mohammad Abboush
Second Examiner	apl. Prof. Dr. Christoph Knieke
Submission Date	February 27, 2026

Declaration of Authorship

I hereby declare that I have composed the thesis independently, have not used any sources or aids other than those indicated, and that all parts of the thesis that have been quoted or summarized from other sources are marked as such. I also confirm that the thesis has neither been submitted in this form nor in a similar form to any other examination authority.

Furthermore, I declare that I do not consent to the public availability of my thesis in the institute's and/or university library. I also agree to a software-based plagiarism check.

Clausthal-Zellerfeld, February 22, 2026

Location, Date	First name and family name

Contents

Declaration of Authorship i

Table of Contents vi

List of Abbreviations vii

List of Tables ix

Abstract xi

Zusammenfassung xiii

1 Introduction 1

1.1 Context and Motivation 1
1.2 Research Questions 2
1.3 Related Work 3
1.4 Solution 5
1.5 Contributions 6
1.6 Structure 7

2 Background 9

2.1 Model-Based Development and Testing Phases 9
2.1.1 Model-in-the-Loop (MIL) 9
2.1.2 Software-in-the-Loop (SIL) 10
2.1.3 Processor-in-the-Loop (PIL) 10
2.1.4 Hardware-in-the-Loop (HIL) 10
2.1.5 Vehicle-in-the-Loop (VIL) 11
2.2 The Real Test Drive Validation Process 11
2.2.1 Test Drive Data Acquisition 11
2.2.2 Data Volume and Analysis Challenges 12
2.2.3 The A2D2 Dataset 12
2.3 Fault Injection Approaches 13
2.3.1 Fault Types 13

2.3.2 Fault Locations 14
2.4 Deep Learning for Time-Series Data 15
2.4.1 Convolutional Neural Networks (CNNs) 15
2.4.2 Batch Normalization 15
2.4.3 Pooling Operations 16
2.5 Self-Supervised Learning 16
2.6 Contrastive Learning 17
2.6.1 Problem Formulation 17
2.6.2 Data Augmentation 18
2.6.3 Encoder and Projection Head 18
2.6.4 Cosine Similarity 19
2.6.5 NT-Xent Loss 19
2.6.6 Role of the Temperature Parameter 20
2.6.7 Relationship Between Batch Size and Negative Examples 21
2.6.8 Connection to Noise-Contrastive Estimation 21
2.7 Anomaly Detection 21
2.7.1 Distance-Based Anomaly Detection 21
2.7.2 Threshold-Based Detection 22
2.8 Summary 22

3 Methodology 25
3.1 Problem Formulation 26
3.2 Self-Supervised Contrastive Learning with SimCLR 27
3.2.1 The Contrastive Learning Paradigm 27
3.2.2 Data Augmentation Design 27
3.2.3 Encoder Architecture 29
3.2.4 Projection Head 29
3.2.5 NT-Xent Loss 30
3.2.6 Optimisation Objective 31
3.3 Healthy Reference Modeling 31
3.4 Similarity-Based Anomaly Detection 31
3.5 Cross-Domain Transfer Setting 32
3.6 Summary 33

4 Implementation and Experimental Setup 35
4.1 Datasets 35
4.1.1 A2D2 Training Data 35
4.1.2 HIL Calibration and Test Data 37

4.2 Data Preprocessing Pipeline 39
4.2.1 Normalization 39
4.2.2 Sliding Window Segmentation 39
4.2.3 Dataset Fusion Implementation 40
4.3 SimCLR Training Implementation 41
4.3.1 Data Augmentation Modules 41
4.3.2 Encoder Architecture Implementation 42
4.3.3 Projection Head Implementation 43
4.3.4 Hyperparameter Configuration 43
4.3.5 Training Loop Implementation 44
4.4 Calibration Implementation 45
4.5 Fault Detection Evaluation Protocol 46
4.5.1 Per-Fault Evaluation 46
4.5.2 Binary Classification Evaluation 47
4.5.3 ROC and Precision-Recall Curves 47
4.5.4 Reproducibility Measures 48
4.6 Summary 48

5 Results and Discussion 51
5.1 Training Results 51
5.1.1 SimCLR Training Summary 51
5.2 Multi-Threshold Fault Detection Results 53
5.2.1 Results at 15th Percentile Threshold 53
5.2.2 Results at 20th Percentile Threshold 54
5.2.3 Results at 25th Percentile Threshold 54
5.2.4 Results at 30th Percentile Threshold 55
5.2.5 Results at 35th Percentile Threshold 55
5.2.6 Results at 40th Percentile Threshold 56
5.2.7 Cross-Threshold Comparison 56
5.2.8 Per-Fault-Type Recall Heatmap 57
5.3 Binary Classification: Healthy vs. Faulty 58
5.3.1 Binary Confusion Matrices 59
5.3.2 Binary Classification Metrics 59
5.3.3 ROC Curves 60
5.4 Similarity Distribution Analysis 61
5.5 Computing Cost Analysis 61
5.5.1 Discussion of Computing Costs 62

5.6 Discussion 63
5.6.1 Effectiveness of SSL for Automotive Fault Detection . . . 63
5.6.2 Domain Alignment: A2D2 to HIL 63
5.6.3 Impact of Threshold Selection 64
5.6.4 Fault Type Detectability 64
5.6.5 Limitations 65

6 Conclusion and Future Work 67
6.1 Conclusion 67
6.1.1 Key Contributions 68
6.2 Future Work 69

Bibliography 73

List of Abbreviations

HIL Hardware-in-the-Loop

A2D2 Audi Autonomous Driving Dataset

SimCLR Simple Contrastive Learning of Representations

NT-Xent Normalized Temperature-scaled Cross-Entropy

ROC Receiver Operating Characteristic

AUC Area Under the Curve

SSL Self-Supervised Learning

ABS Anti-lock Braking System

List of Tables

2.1 Mathematical representations of signal fault types used in this thesis. 13

4.1 A2D2 JSON files used for training, with city and road type classification. 37
4.2 HIL data files used for calibration and testing. 38
4.3 Layer-by-layer implementation of the 1D-CNN encoder. 42
4.4 Final hyperparameter configuration for SimCLR training. 44

5.1 SimCLR training summary. 52
5.2 Fault detection results at the 15th percentile threshold ($\tau = 0.8901$). 53
5.3 Fault detection results at the 20th percentile threshold ($\tau = 0.9268$). 54
5.4 Fault detection results at the 25th percentile threshold ($\tau = 0.9477$). 54
5.5 Fault detection results at the 30th percentile threshold ($\tau = 0.9742$). 55
5.6 Fault detection results at the 35th percentile threshold ($\tau = 0.9786$). 55
5.7 Fault detection results at the 40th percentile threshold ($\tau = 0.9803$). 56
5.8 Cross-threshold comparison of overall detection performance. . . 56
5.9 Binary classification results (healthy vs. faulty) across all thresholds. 59
5.10 Computing cost analysis for all pipeline stages (CPU execution). 62

Abstract

The evolution of Automotive Software Systems (ASSs) has led to growing complexity in sensor networks, increasing the risk of undetected faults that can compromise vehicle safety. Traditional fault detection methods depend on supervised learning with large labeled datasets, which are expensive to obtain and inherently incomplete—they cannot cover every possible failure mode.

This thesis proposes a self-supervised contrastive learning framework for automotive sensor fault detection that requires no labeled fault data. A 1D Convolutional Neural Network encoder is trained using the SimCLR framework on healthy driving data from the Audi Autonomous Driving Dataset (A2D2), learning to map time-series sensor windows into a 256-dimensional embedding space. Anomalies are detected by measuring the cosine similarity between test embeddings and a healthy reference centroid computed from a small calibration set of Hardware-in-the-Loop (HIL) data.

The framework is evaluated on six fault scenarios—gain, noise, and stuck-at faults injected into accelerator pedal and vehicle speed sensors—across six detection threshold percentiles (15th through 40th). Comprehensive metrics including precision, recall, F1-score, accuracy, confusion matrices, and ROC curves are reported for all experiments, alongside computing cost measurements for training and inference.

Results demonstrate that the framework successfully learns robust representations from real driving data without labels, transfers this knowledge to HIL data where faults are known, and enables anomaly detection across domains and scenarios. The multi-threshold analysis quantifies the precision–recall trade-off, providing practical guidance for threshold selection in safety-critical applications.

Keywords: Self-Supervised Learning, Contrastive Learning, SimCLR, Automotive Sensor Fault Detection, Hardware-in-the-Loop, Anomaly Detection, 1D-CNN, Transfer Learning

Zusammenfassung

Die zunehmende Komplexität moderner Fahrzeugsoftwaresysteme erhöht das Risiko unerkannter Sensorfehler, die die Fahrzeugsicherheit gefährden können. Herkömmliche Fehlererkennungsmethoden basieren auf überwachtem Lernen mit großen gelabelten Datensätzen, deren Beschaffung kostenintensiv und inhärent unvollständig ist.

Diese Arbeit schlägt ein selbstüberwachtes kontrastives Lernverfahren zur Fehlererkennung bei Fahrzeugsensoren vor, das keine gelabelten Fehlerdaten benötigt. Ein 1D-CNN-Encoder wird mittels des SimCLR-Frameworks ausschließlich auf gesunden Fahrdaten des Audi Autonomous Driving Dataset (A2D2) trainiert. Anomalien werden durch Kosinusähnlichkeit der Einbettungen zu einem gesunden Referenz-Zentroid erkannt, der aus einer kurzen Kalibrierungsaufnahme von Hardware-in-the-Loop-Daten berechnet wird.

Die Ergebnisse zeigen, dass das Framework erfolgreich robuste Repräsentationen aus realen Fahrdaten ohne Labels lernt, dieses Wissen auf HiL-Daten mit bekannten Fehlern überträgt und eine Anomalieerkennung über verschiedene Domänen und Szenarien hinweg ermöglicht.

Das Framework wird an sechs Fehlerszenarien evaluiert—Verstärkungsfehler, Rauschfehler und Stuck-at-Fehler in Gaspedalstellung und Fahrzeuggeschwindigkeit— über sechs Schwellenwert-Perzentile (15. bis 40.). Präzision, Recall, F1-Score, Genauigkeit, Konfusionsmatrizen und ROC-Kurven werden für alle Experimente berichtet, zusammen mit den Berechnungskosten für Training und Inferenz.

1 Introduction

1.1 Context and Motivation

Modern vehicles rely on a growing network of Electronic Control Units (ECUs) that continuously process data from dozens of sensors to govern safety-critical functions such as engine management, braking, and steering (Abboush et al., 2022). A single mid-range passenger car today can contain more than 70 ECUs communicating over Controller Area Network (CAN) buses, each dependent on accurate sensor readings to maintain safe operation. When a sensor produces faulty measurements—whether due to electrical interference, mechanical degradation, or component failure—the downstream control logic may issue incorrect commands, potentially leading to hazardous situations for occupants and other road users.

The ISO 26262 standard for functional safety of road vehicles mandates rigorous testing and validation of these systems, including Hardware-in-the-Loop (HIL) simulation and real-world test drives (*ISO 26262: Road Vehicles – Functional Safety* 2018). During such validation campaigns, vast quantities of time-series sensor data are recorded, often exceeding several gigabytes per test session. Manual inspection of this data by domain experts is not only time-consuming but also prone to human oversight, particularly for subtle or intermittent faults that may not produce immediately obvious signal distortions.

Artificial intelligence offers a path toward automated fault detection. Supervised deep learning methods, including Convolutional Neural Networks (CNNs) combined with Gated Recurrent Units (GRUs), have demonstrated strong performance in classifying fault types and locations within automotive systems (Ghannoum, 2025; Abboush et al., 2022). However, these approaches depend on large, labeled datasets that include examples of every fault type the model is expected to recognize. Collecting such labeled fault data requires deliberate fault

injection campaigns and manual annotation—a process that is expensive, time-intensive, and inherently incomplete, since it cannot cover every conceivable fault mode that may arise in practice.

Self-supervised learning (SSL) offers a fundamentally different paradigm. Rather than learning from labeled examples, SSL methods extract supervisory signals from the structure of the data itself (Jaiswal et al., 2021). In the context of fault detection, this means a model can learn what *normal* sensor behavior looks like by training exclusively on real driving data, which is abundantly available from routine test drives. Any subsequent deviation from the learned normal patterns can then be flagged as a potential anomaly, without ever having seen a single labeled fault example during training.

Among SSL approaches, contrastive learning has emerged as particularly effective for representation learning (T. Chen et al., 2020). The core principle is to train an encoder network such that similar inputs (augmented views of the same data sample) produce similar representations, while dissimilar inputs produce distant representations. SimCLR (ibid.), one of the most widely adopted contrastive frameworks, has demonstrated state-of-the-art results across diverse domains, yet its application to automotive sensor fault detection remains largely unexplored.

This thesis investigates whether contrastive self-supervised learning, trained solely on healthy driving data from the Audi Autonomous Driving Dataset (A2D2) (Audi AG, 2020), can effectively detect sensor faults injected through a HIL simulation environment. The work addresses a practical scenario: a model trained on real-world driving data from one vehicle is deployed to detect faults in a different data acquisition system, testing the cross-domain generalization capability of the learned representations.

1.2 Research Questions

This thesis addresses the following research questions:

- **RQ1:** Can a contrastive self-supervised learning model, trained exclusively on healthy sensor data from real-world driving recordings, reliably detect sensor faults without any labeled fault examples?

- **RQ2:** How does the selection of the anomaly detection threshold affect the trade-off between detection sensitivity (recall) and false alarm rate (precision), and what threshold range is most suitable for safety-critical automotive applications?
- **RQ3:** Which types of sensor faults (gain, noise, stuck-at) are most and least detectable by the proposed approach, and what physical factors explain the observed differences?

1.3 Related Work

This section reviews prior research at the intersection of self-supervised learning, contrastive representation learning, anomaly detection, and automotive fault diagnosis. The objective is to situate the proposed method within the existing literature and to identify the specific gaps that this thesis addresses.

T. Zhang et al. (2022) survey self-supervised and data-efficient learning methods for machinery fault diagnosis and conclude that contrastive learning is the most effective paradigm when labelled data are scarce. Their review reveals that the vast majority of SSL-based fault diagnosis studies focus on benchmark bearing datasets; automotive sensor systems remain almost entirely unexplored.

Ding et al. (2022) employ contrastive predictive coding for incipient bearing fault detection, capturing temporal dependencies that enable early fault identification with minimal supervision. Although label efficiency is improved, the approach still requires a small labelled set for fine-tuning and has not been validated on automotive sensor signals. Similarly, X. Chen et al. (2023) survey domain adaptation and transfer learning methods for bearing fault diagnosis, demonstrating that representations learned in one operating condition can transfer to another—a scenario analogous to the A2D2-to-HIL transfer addressed in this thesis. Again, labelled examples are required for classifier adaptation, and the methods are designed for vibration data rather than for accelerator or speed sensor signals.

X. Zhang et al. (2022) propose a self-supervised contrastive framework that leverages time-frequency consistency for time-series representation learning. By enforcing agreement between temporal and spectral views of the same signal, the method learns representations that capture both local and global signal characteristics. While this represents a substantial advance in SSL for time

series, the dependency on domain-specific frequency transforms limits direct applicability to non-stationary automotive signals.

Eldele et al. (2021) introduce temporal and contextual contrasting (TS-TCC) that learns robust temporal representations through paired weak-strong augmentations; the learned representations transfer well to downstream classification tasks. The evaluation, however, remains confined to standard benchmarks rather than safety-critical automotive systems.

Among contrastive learning frameworks, SimCLR (T. Chen et al., 2020) established that strong visual representations can be learned by maximising agreement between augmented views of the same sample while minimising agreement with views of other samples. Its core components—data augmentation, a base encoder, a projection head, and the NT-Xent loss—are architecture-agnostic and have inspired numerous time-series adaptations. MoCo (He et al., 2020) addresses SimCLR's batch-size sensitivity by maintaining a momentum-updated dictionary of negative examples, while BYOL (Grill et al., 2020) eliminates the need for negative pairs entirely. For temporal data, TS2Vec (Yue et al., 2022) performs contrastive learning at multiple temporal scales, and TS-TCC (Eldele et al., 2021) introduces paired weak-strong augmentations. Despite these advances, no prior work has systematically adapted SimCLR to automotive sensor fault detection, nor has any study evaluated its ability to transfer representations from real driving data to hardware-in-the-loop simulated faults.

Ruff et al. (2021) provide a theoretical foundation for deep anomaly detection, demonstrating that self-supervised pretraining followed by simple distance-based scoring in the learned feature space consistently outperforms end-to-end trained detectors. Pang et al. (2021) identify the core challenges of learning effective anomaly representations without labelled anomalies and of detecting complex, multi-feature faults. Hojjati, Ho, and Armanfard (2024) confirm that contrastive embeddings naturally separate in-distribution from out-of-distribution samples, supporting the two-stage pipeline adopted in this thesis: contrastive pretraining on healthy data only, followed by cosine similarity thresholding.

Within the automotive domain, Matos et al. (2024) review sensor failures in autonomous vehicles and explicitly identify label scarcity as the primary barrier to adoption of learning-based detection methods; they recommend self-supervised and transfer learning as key future directions. Abboush et al. (2022) and Ghannoum (2025) propose supervised CNN-GRU frameworks that achieve

high classification accuracy on HIL-simulated faults. Both methods, however, require labelled examples of every fault type and location. Neither operates in an unsupervised setting. Chiaroni et al. (2021) review SSL applications in automotive perception, confirming the viability of contrastive methods in the automotive domain but noting that fault detection remains an open application area.

A synthesis of the reviewed literature reveals six recurring research gaps:

- SSL-based fault diagnosis remains concentrated on rotating machinery; automotive sensor systems are neglected.
- Existing SSL methods reduce but do not eliminate the need for labelled fault data (fine-tuning, semi-supervised classification).
- Cross-domain transfer between real and simulated acquisition systems has not been systematically evaluated for SSL-based automotive fault detection.
- Detection performance is typically reported at a single operating point, ignoring the precision–recall trade-off controlled by threshold selection.
- Many studies report only accuracy, omitting precision, recall, F1-score, and ROC analysis essential for safety-critical applications.
- Computational costs of training and inference are rarely reported, obscuring practical feasibility.

This thesis directly addresses all six gaps. It applies contrastive SSL to automotive accelerator and speed sensors, requires *zero* labelled fault data, validates A2D2-to-HIL transfer, evaluates six threshold settings, reports comprehensive metrics for every experiment, and measures computational costs for all pipeline stages. In doing so, to the best of our knowledge, it provides the first demon stration of fully unsupervised, cross-domain automotive sensor fault detection using self-supervised contrastive learning.

1.4 Solution

The proposed framework operates in three distinct phases:

1. **Self-supervised pretraining.** A one-dimensional CNN encoder is trained on the SimCLR contrastive learning objective using healthy driving data from the A2D2 dataset. The encoder learns to map time-series sensor windows into a compact embedding space where semantically similar windows are placed close together. Three domain-specific augmentation strategies—Gaussian jittering, amplitude scaling, and temporal masking—generate the positive pairs required for contrastive training.

2. **Anomaly detection calibration.** A small quantity of healthy HIL data is passed through the frozen encoder to establish a reference distribution in the embedding space. The centroid of the healthy embeddings and the distribution of cosine similarities to that centroid are computed, and detection thresholds are set at multiple percentiles of this distribution.

3. **Fault detection and evaluation.** HIL data containing injected faults (gain, noise, and stuck-at types across accelerator pedal and vehicle speed sensors) is evaluated against the calibrated thresholds. Performance is assessed using precision, recall, F1-score, accuracy, confusion matrices, and Receiver Operating Characteristic (ROC) curves across multiple threshold settings.

1.5 Contributions

The primary objective of this thesis is to develop, implement, and evaluate a self-supervised contrastive learning framework for automotive sensor fault detection that satisfies the following requirements:

1. Requires no labeled fault data during any phase of training or calibration.

2. Design a Self-Supervised Learning (SSL) pipeline that:

   - Learns robust representations from real driving data (without labels),

   - Transfers learned representations to HIL data (where faults are known),

   - Enables anomaly detection across domains and scenarios.

3. Provides a systematic multi-threshold analysis (15th through 40th percentile) to quantify the sensitivity–specificity trade-off.

4. Demonstrates cross-domain transfer from real-world A2D2 data to HIL data.

5. Reports all evaluation metrics mandated for safety-critical systems: precision, recall, F1-score, accuracy, confusion matrices, and ROC curves with AUC values.

6. Documents computational costs for training and inference to assess deployment feasibility.

1.6 Structure

The remainder of this thesis is organized as follows:

- **Chapter 2: Background.** Introduces the theoretical foundations, including model-based development and testing phases in the automotive V-model, the real test drive validation process, fault injection approaches, contrastive self-supervised learning with detailed mathematical formulations, and anomaly detection methods.

- **Chapter 3: Methodology.** Presents the theoretical foundations and methodological design of the proposed self-supervised fault detection framework, including problem formulation, contrastive learning with SimCLR, domain-appropriate data augmentations, 1D-CNN encoder architecture, healthy reference modeling via centroid estimation, similarity-based anomaly detection, evaluation metrics, and the cross-domain transfer setting.

- **Chapter 4: Implementation and Experimental Setup.** Describes the concrete realization of the proposed methodology, including A2D2 and HIL dataset processing, normalization and windowing pipelines, the 1D-CNN encoder and projection head implementation, hyperparameter configuration, training procedure with SimCLR, calibration of the healthy centroid and percentile thresholds, and the evaluation protocol for both per-fault and binary classification. Computational environment, reproducibility measures, and intermediate validation results are also documented.

- **Chapter 5: Results and Discussion.** Reports the full evaluation results, including per-fault detection performance across six thresholds, binary classification results (healthy vs. faulty), ROC analysis, per-fault-type analysis, and computing cost measurements. A detailed discussion interprets the findings in the context of practical automotive deployment.
- **Chapter 6: Conclusion and Future Work.** Summarizes the contributions, answers the research questions, and outlines directions for future research.

2 Background

This chapter introduces the theoretical foundations required to understand the proposed fault detection framework. It covers the automotive development and validation process, fault injection techniques used in HIL testing, the fundamentals of deep learning for time-series data, and—most critically—a detailed treatment of contrastive self-supervised learning with full mathematical formulations.

2.1 Model-Based Development and Testing Phases

Model-Based Development (MBD) has become the standard methodology in the automotive industry for designing and validating embedded software systems. MBD uses mathematical and simulation models to facilitate early design validation and verification, following the V-model development lifecycle (Abboush et al., 2022). The left side of the V-model represents the design and development stages, while the right side represents the corresponding verification and validation stages, progressing from abstract models to physical hardware. Figure 2.1 illustrates this lifecycle.

2.1.1 Model-in-the-Loop (MIL)

Model-in-the-Loop testing validates the system design within a purely virtual environment. Both the controller and the plant are represented as mathematical models, typically developed in tools such as MATLAB/Simulink. MIL enables early verification of algorithmic correctness, control strategy logic, and functional behavior without requiring any hardware components. Because MIL testing is entirely software-based, it offers the fastest iteration cycles and the lowest cost among all V-model testing phases.

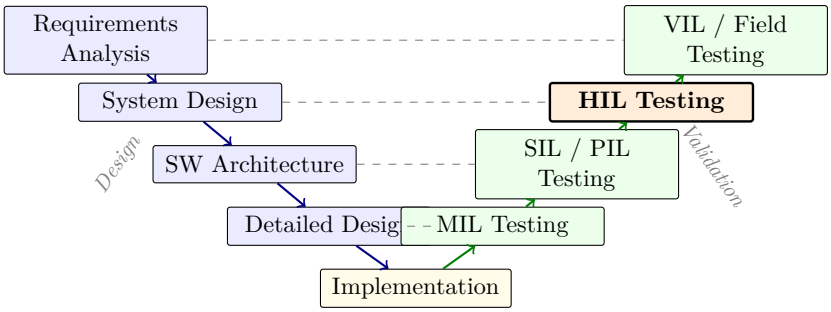

Figure 2.1: The automotive V-model development lifecycle. The left arm represents successive design refinement phases; the right arm represents the corresponding verification and validation phases. Dashed lines indicate the correspondence between design and test levels. HIL testing (highlighted) is the primary validation phase used in this thesis.

2.1.2 Software-in-the-Loop (SIL)

In SIL testing, the controller model is replaced by the actual generated source code, which runs on the host development computer against the simulated plant model. SIL testing verifies that the auto-generated code behaves identically to the original controller model, identifying discrepancies that may arise from code generation, numerical precision, or implementation-specific behavior.

2.1.3 Processor-in-the-Loop (PIL)

PIL testing executes the controller software on the target microprocessor or ECU hardware, while the plant model still runs in simulation. This phase captures processor-specific effects such as fixed-point arithmetic, limited memory, interrupt handling, and execution timing that are invisible during SIL testing.

2.1.4 Hardware-in-the-Loop (HIL)

HIL testing represents a critical validation stage where the complete ECU hardware, including its physical interfaces, runs against a real-time simulation of the vehicle plant model (Waeltermann, 2016). The HIL simulator generates realistic electrical signals that mimic the outputs of real sensors (e.g., accelerometer readings, wheel speed pulses, pedal position voltages) and receives the ECU's actuator commands in return. This closed-loop configuration allows validation

of the entire hardware-software stack under conditions that closely approximate real vehicle operation, but in a safe, repeatable laboratory environment.

HIL testing is particularly valuable for fault injection studies because faults can be introduced systematically at precisely defined times, locations, and magnitudes—something that would be impractical or dangerous to do in a real vehicle. The dSPACE HIL platform, used in the research group at TU Clausthal, provides Configuration Desk for system setup and Control Desk for real-time monitoring and data logging during test execution (Abboush et al., 2022).

2.1.5 Vehicle-in-the-Loop (VIL)

VIL testing places a physical vehicle—or a high-fidelity driving simulator—within a virtual environment that generates realistic driving scenarios. The vehicle's control systems respond to simulated road conditions, traffic, and weather, while the virtual environment adapts in real time to the vehicle's actions. VIL bridges the gap between laboratory HIL testing and real-world test drives.

2.2 The Real Test Drive Validation Process

The real test drive constitutes the final validation stage for automotive systems before production release. During these drives, the vehicle operates under actual road conditions while onboard data acquisition systems continuously record sensor measurements at high sampling rates. This section describes the process as it relates to the data used in this thesis.

2.2.1 Test Drive Data Acquisition

During a real test drive, data acquisition systems record signals from all relevant sensors and ECUs over the vehicle's CAN bus. Typical recorded signals include:

- Accelerator pedal position (%), reflecting the driver's throttle input
- Vehicle speed (km/h), measured by wheel speed sensors
- Engine speed (RPM), from the crankshaft position sensor

- Steering wheel angle (degrees), from the steering column sensor
- Brake pedal position (%), indicating braking intensity
- Lateral and longitudinal acceleration (m/s^2), from inertial measurement units

2.2.2 Data Volume and Analysis Challenges

A single test drive of 30 minutes at 100 Hz sampling generates over 180,000 data points per sensor channel. With multiple sensors recorded simultaneously, the total data volume per test session can reach several gigabytes. Manual expert analysis of this volume is impractical, motivating the development of automated analysis methods.

The key challenge is that faults in real-world data are rare, unlabeled, and diverse. A sensor may produce subtly incorrect readings for only a few seconds within hours of normal operation. Traditional threshold-based monitoring catches only the most severe deviations, while intermittent or slowly drifting faults may go undetected. This motivates the self-supervised approach developed in this thesis, which learns a comprehensive model of normal behavior and can identify even subtle deviations.

2.2.3 The A2D2 Dataset

The Audi Autonomous Driving Dataset (A2D2) (Audi AG, 2020) is a large-scale public dataset recorded by Audi AG for research in autonomous driving. It contains sensor data from multiple test drives conducted in three German cities: Gaimersheim, Ingolstadt, and Munich. The dataset includes camera images, LiDAR point clouds, and CAN bus signals recorded during real-world driving.

For the purpose of this thesis, only the CAN bus signals are used, specifically the accelerator pedal position and vehicle speed. These signals provide a representative pair of automotive sensors with a clear physical relationship: the accelerator pedal is a driver input, and the vehicle speed is the corresponding system output.

In the Audi A2D2 dataset used in this thesis, the accelerator pedal signal is recorded at approximately 100 Hz, while the vehicle speed signal is recorded at approximately 50 Hz. These different rates reflect the physical characteristics of each measurement: the accelerator pedal position can change rapidly with the driver's foot movement, while vehicle speed changes more gradually due to the vehicle's inertia.

The data is stored in JSON format with microsecond-resolution timestamps, enabling precise sampling rate analysis and signal alignment.

2.3 Fault Injection Approaches

Fault injection is a validation technique used to assess how a system behaves in the presence of faults (Abboush et al., 2022). ISO 26262 recommends fault injection at various stages of the V-model to verify that safety mechanisms correctly detect and mitigate faults. A fault injection scenario is characterized by three attributes: *when* the fault occurs (fault time), *where* it occurs (fault location), and *what type* of fault is introduced (fault type).

2.3.1 Fault Types

Table 2.1 summarizes the three fault types used in this thesis, along with their mathematical representations and typical physical causes.

Table 2.1: Mathematical representations of signal fault types used in this thesis.

Fault Type	**Mathematical Representation**	**Physical Cause**
Healthy signal	$y(t) = x(t)$	Normal operation
Gain fault	$y(t) = \alpha \cdot x(t), \quad \alpha \neq 1$	Calibration error, amplifier drift
Noise fault	$y(t) = x(t) + \eta(t), \quad \eta \sim \mathcal{N}(0, \sigma^2)$	EMI, sensor degradation
Stuck-at fault	$y(t) = c \quad \forall\, t \geq t_f$	Broken wiring, component failure

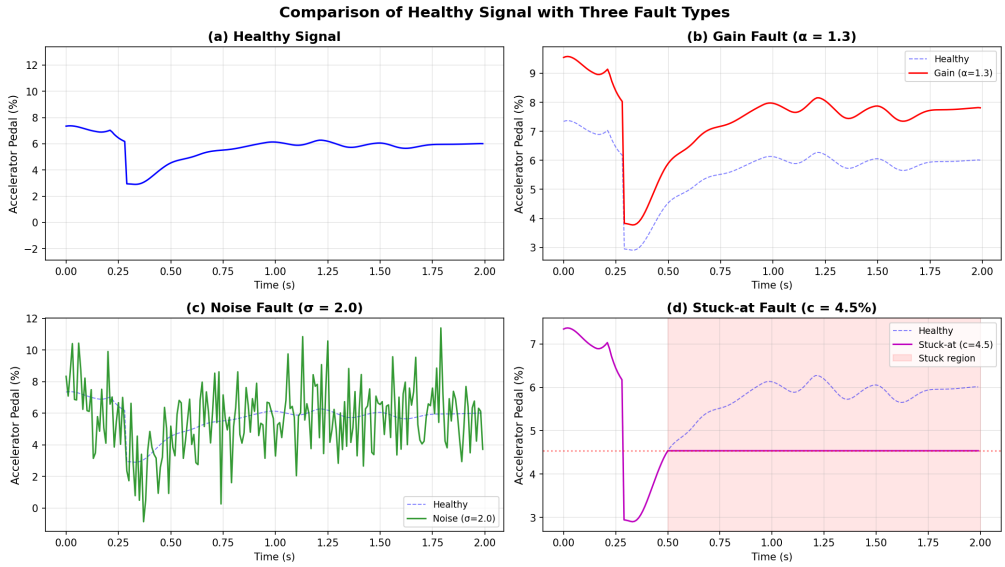

Figure 2.2: Comparison of a healthy sensor signal with three fault types: gain, noise, and stuck-at.

Gain fault. The sensor output is scaled by an incorrect factor α. When $\alpha > 1$, the signal is amplified; when $\alpha < 1$, it is attenuated. This fault typically arises from calibration errors in the signal conditioning circuit or from amplifier drift due to temperature changes. The distortion is proportional to the signal magnitude, meaning it is most visible during dynamic driving conditions and nearly invisible when the true value is close to zero.

Noise fault. Additive Gaussian noise $\eta(t)$ corrupts the sensor output. The noise standard deviation σ determines the fault severity. Noise faults are caused by electromagnetic interference (EMI) from nearby components, degraded shielding, or loose electrical connections. Unlike gain faults, noise faults are visible regardless of the signal magnitude, appearing as high-frequency fluctuations superimposed on the true signal.

Stuck-at fault. The sensor output freezes at a constant value c from the fault onset time t_f onward. This represents the most severe fault type, corresponding to a complete sensor failure such as a broken wire, shorted circuit, or failed sensing element. The frozen value may be zero, the last valid reading, or an arbitrary voltage determined by the failure mode.

2.3.2 Fault Locations

In this thesis, faults are injected into two sensor signals:

1. **Accelerator pedal position (%):** Measures the driver's throttle demand. Range: 0% (released) to 100% (fully pressed). A fault in this sensor can cause unintended acceleration or loss of throttle response.

2. **Vehicle speed (km/h):** Measures how fast the vehicle is traveling. A fault in this sensor can affect cruise control, ABS, and stability control systems.

The combination of three fault types and two fault locations yields six distinct fault scenarios, each evaluated independently in Chapter 5.

2.4 Deep Learning for Time-Series Data

This section introduces the deep learning architectures relevant to the encoder network used in this thesis.

2.4.1 Convolutional Neural Networks (CNNs)

A CNN applies learnable convolutional filters to extract local patterns from input data (LeCun, Bengio, and Hinton, 2015). For one-dimensional time-series data, a 1D convolutional layer computes:

$$z_j^{(l)}[n] = \sigma\left(\sum_{i=1}^{C_{l-1}} \sum_{m=0}^{K-1} w_{ij}^{(l)}[m] \cdot a_i^{(l-1)}[n \cdot s + m] + b_j^{(l)}\right) \tag{2.1}$$

where $a_i^{(l-1)}[n]$ is the i-th input channel at position n, $w_{ij}^{(l)}[m]$ are the filter weights of length K, s is the stride, $b_j^{(l)}$ is the bias term, $\sigma(\cdot)$ is a nonlinear activation function, and $z_j^{(l)}[n]$ is the j-th output feature map at position n. The number of input channels is C_{l-1} and the number of output channels (filters) is C_l.

2.4.2 Batch Normalization

Batch normalization (Ioffe and Szegedy, 2015) normalizes the pre-activation values within each mini-batch to stabilize and accelerate training:

$$\hat{z}_j = \frac{z_j - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \cdot \gamma + \beta \tag{2.2}$$

where $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}^2$ are the mean and variance computed over the current mini-batch, γ and β are learnable scale and shift parameters, and ϵ is a small constant for numerical stability.

2.4.3 Pooling Operations

Max pooling selects the maximum value within each pooling window of size p:

$$a_j^{\text{pool}}[n] = \max_{m=0}^{p-1} z_j[n \cdot p + m] \tag{2.3}$$

Global average pooling computes the mean across the entire temporal dimension:

$$\bar{a}_j = \frac{1}{T} \sum_{n=1}^{T} z_j[n] \tag{2.4}$$

producing a single scalar per feature map. This operation converts variable-length feature maps into a fixed-size representation, which is essential for mapping time-series windows of arbitrary length to fixed-dimensional embeddings.

2.5 Self-Supervised Learning

Self-supervised learning (SSL) is a paradigm in which a model learns useful representations from unlabeled data by solving pretext tasks that are derived from the data's own structure (Jaiswal et al., 2021). The supervisory signal comes not from human annotations but from the inherent properties of the data—temporal order, spatial coherence, or augmentation invariance.

SSL methods can be categorized into three families:

1. **Generative methods** learn to reconstruct the input data (e.g., autoencoders, variational autoencoders). The reconstruction objective encourages the model to capture the most informative features of the data.

2. **Predictive methods** learn to predict a missing or future part of the input from the available context (e.g., masked language models, Contrastive Predictive Coding (Oord, Li, and Vinyals, 2018)).

3. **Contrastive methods** learn to distinguish between similar and dissimilar data pairs in a learned embedding space (T. Chen et al., 2020; He et al., 2020). This thesis employs contrastive learning.

The advantage of SSL for fault detection is that it requires only healthy data for training. Since healthy sensor recordings are abundantly available from routine test drives and HIL experiments, SSL eliminates the need for expensive labeled fault datasets.

2.6 Contrastive Learning

Contrastive learning trains an encoder to produce representations in which similar samples are close together and dissimilar samples are far apart. This section provides the mathematical formulation in detail, as the contrastive objective is the central training mechanism of the proposed framework.

2.6.1 Problem Formulation

Given an unlabeled dataset $\mathcal{D} = \{x_1, x_2, \ldots, x_N\}$ of N sensor data windows, the goal is to learn an encoder function $f_\theta : \mathcal{X} \to \mathbb{R}^d$ parameterized by θ that maps each input $x_i \in \mathcal{X}$ to a d-dimensional representation $h_i = f_\theta(x_i)$ such that the representation captures the essential structure of the data.

To achieve this without labels, contrastive learning constructs *positive pairs* (two views of the same sample that should have similar representations) and *negative pairs* (views of different samples that should have dissimilar representations).

2.6.2 Data Augmentation

A stochastic augmentation module $\mathcal{T}$ transforms each sample x_i into two correlated views:

$$\tilde{x}_i = t(x_i), \quad t \sim \mathcal{T} \tag{2.5}$$

$$\tilde{x}_j = t'(x_i), \quad t' \sim \mathcal{T} \tag{2.6}$$

where t and t' are independently sampled augmentation functions from the family $\mathcal{T}$. The pair $(\tilde{x}_i, \tilde{x}_j)$ forms a *positive pair*: two different views of the same underlying data that the encoder should map to similar representations.

For time-series data, appropriate augmentations must preserve the semantic content of the signal while introducing sufficient variability. Three augmentation strategies are employed in this thesis (detailed in Chapter 4): Gaussian jittering, amplitude scaling, and temporal masking.

2.6.3 Encoder and Projection Head

The contrastive framework consists of two components, as illustrated in Figure 2.3:

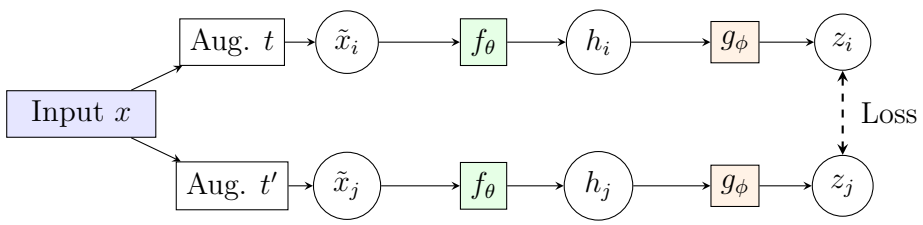

Figure 2.3: The Contrastive Learning Framework (SimCLR) architecture. Two augmented views of the same input are processed through a shared encoder and projection head to minimize the NT-Xent loss.

1. An encoder $f_\theta(\cdot)$ that extracts a representation vector from each augmented view:

$$h_i = f_\theta(\tilde{x}_i) \in \mathbb{R}^d \tag{2.7}$$

2. A projection head $g_\phi(\cdot)$ that maps the representation to a lower-dimensional space where the contrastive loss is applied:

$$z_i = g_\phi(h_i) = W^{(2)} \cdot \sigma\Big(W^{(1)} h_i + b^{(1)}\Big) + b^{(2)} \tag{2.8}$$

   where $W^{(1)} \in \mathbb{R}^{d \times d}$, $W^{(2)} \in \mathbb{R}^{p \times d}$ are weight matrices, $b^{(1)}, b^{(2)}$ are bias vectors, $\sigma(\cdot)$ is the ReLU activation, and p is the projection dimension.

A critical finding of T. Chen et al. (2020) is that the contrastive loss should be applied to the projected representations z_i rather than the encoder representations h_i, because the projection head can discard information (such as augmentation-specific details) that is useful for the contrastive task but irrelevant for downstream tasks. After training, the projection head is discarded, and only the encoder representations h_i are used.

2.6.4 Cosine Similarity

The similarity between two projected representations is measured using cosine similarity:

$$\mathrm{sim}(z_i, z_j) = \frac{z_i^\top z_j}{\|z_i\|_2 \cdot \|z_j\|_2} \tag{2.9}$$

Cosine similarity measures the angular alignment between two vectors, independent of their magnitudes. It ranges from -1 (opposite directions) through 0 (orthogonal) to $+1$ (identical direction). This metric is preferred over Euclidean distance in contrastive learning because it is invariant to the scale of the representations, which simplifies optimization.

2.6.5 NT-Xent Loss

The Normalized Temperature-scaled Cross-Entropy (NT-Xent) loss, introduced by T. Chen et al. (ibid.), is the training objective of SimCLR. For a mini-batch of N samples, $2N$ augmented views are generated. For a positive pair (i, j) derived from the same original sample, the loss is:

$$\ell_{i,j} = -\log \frac{\exp\Big(\mathrm{sim}(z_i, z_j) \,/\, \tau\Big)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \, \exp\Big(\mathrm{sim}(z_i, z_k) \,/\, \tau\Big)} \tag{2.10}$$

where $\tau > 0$ is a temperature parameter and $\mathbb{1}_{[k \neq i]}$ is an indicator function equal to 1 when $k \neq i$. The numerator contains the similarity of the positive pair, while the denominator sums over all $2N - 1$ other views in the mini-batch, which serve as negative examples.

The total loss over the mini-batch is the average over all positive pairs:

$$\mathcal{L} = \frac{1}{2N} \sum_{k=1}^{N} \Big[\ell_{2k-1,\,2k} + \ell_{2k,\,2k-1} \Big] \tag{2.11}$$

Each original sample contributes two loss terms (one for each view serving as the anchor), ensuring symmetry.

2.6.6 Role of the Temperature Parameter

The temperature τ in Equation 2.10 controls the sharpness of the similarity distribution in the softmax:

- A **low temperature** (e.g., $\tau = 0.1$) produces a peaked distribution that concentrates the gradient on the hardest negative examples. This encourages fine-grained discrimination but can lead to training instability.

- A **high temperature** (e.g., $\tau = 1.0$) produces a flatter distribution that treats all negatives more equally, yielding smoother gradients but potentially weaker discrimination.

- The value $\tau = 0.5$ used in this thesis represents a balance between hard negative mining and stable optimization, following the recommendations of T. Chen et al. (2020).

2.6.7 Relationship Between Batch Size and Negative Examples

For a mini-batch of size N, each positive pair is contrasted against $2N-2$ negative pairs. Larger batch sizes therefore provide more diverse negative examples, which has been shown to improve the quality of learned representations (ibid.). With the batch size $N = 128$ used in this thesis, each sample is compared against 254 negatives per training step.

2.6.8 Connection to Noise-Contrastive Estimation

The NT-Xent loss can be understood as a form of noise-contrastive estimation (NCE) (Gutmann and Hyvärinen, 2010). NCE trains a model to distinguish between data samples (positives) and noise samples (negatives) drawn from a known distribution. In the contrastive learning setting, the "noise" samples are simply the other augmented views in the mini-batch. The temperature-scaled softmax in Equation 2.10 corresponds to a multi-class classification problem where the model must identify the correct positive among all candidates.

2.7 Anomaly Detection

After contrastive pretraining, the encoder produces embeddings that capture the manifold of normal sensor behavior. Anomaly detection identifies test samples whose embeddings deviate significantly from this normal manifold.

2.7.1 Distance-Based Anomaly Detection

The simplest approach computes the distance (or negative similarity) between a test embedding and a reference point representing the normal distribution. In this thesis, the reference is the centroid of the healthy calibration embeddings:

$$\mu_h = \frac{1}{|\mathcal{H}|} \sum_{h_i \in \mathcal{H}} h_i \tag{2.12}$$

where $\mathcal{H}$ is the set of embeddings computed from healthy calibration windows. The anomaly score for a new window with embedding h_{new} is:

$$s = \mathrm{sim}(h_{\mathrm{new}},\, \mu_h) = \frac{h_{\mathrm{new}}^{\top} \mu_h}{\|h_{\mathrm{new}}\|_2 \cdot \|\mu_h\|_2} \tag{2.13}$$

A low similarity score indicates that the test window is far from the healthy centroid in the embedding space and is therefore more likely to contain a fault.

2.7.2 Threshold-Based Detection

The detection decision is made by comparing the anomaly score against a threshold θ:

$$\mathrm{decision}(h_{\mathrm{new}}) = \begin{cases} \textsc{Faulty} & \text{if } s < \theta \\ \textsc{Healthy} & \text{if } s \geq \theta \end{cases} \tag{2.14}$$

The threshold is set based on a percentile of the healthy similarity distribution. Given the sorted healthy similarities $\{s_1 \leq s_2 \leq \cdots \leq s_{|\mathcal{H}|}\}$, the p-th percentile threshold is:

$$\theta_p = s_{\lfloor p/100 \cdot |\mathcal{H}| \rfloor} \tag{2.15}$$

Lower percentiles yield more permissive thresholds (only the most extreme deviations are flagged), while higher percentiles yield stricter thresholds (more faults are caught at the cost of more false alarms). This thesis evaluates six percentiles: 15, 20, 25, 30, 35, and 40.

2.8 Summary

This chapter has established the theoretical foundations for the proposed framework. The automotive V-model and HIL testing provide the application context and the source of both training and test data. The mathematical formulations of contrastive learning—including data augmentation, the NT-Xent loss, and the role of the temperature parameter—define the self-supervised training objective. Cosine similarity-based anomaly detection with percentile thresholding provides the fault detection mechanism. The evaluation metrics

(precision, recall, F1, accuracy, confusion matrix, ROC/AUC) enable a rigorous assessment of detection performance. These components are combined in the methodology presented in Chapter 3.

3 Methodology

This chapter presents the theoretical foundations and methodological design of the proposed self-supervised fault detection framework. The objective is to define the learning paradigm, mathematical formulation, and detection logic. The system is structured into three conceptual phases: (i) self-supervised representation learning, (ii) healthy reference modeling, and (iii) similarity-based anomaly detection. Each phase is motivated by the constraints of automotive sensor fault diagnosis: absence of labelled fault data, need for cross-domain transfer, and safety-critical recall requirements. Figure 3.1 provides an overview of the complete methodology, explicitly showing the pretraining (including pretext and downstream tasks), domain alignment, fault detection process, and validation.

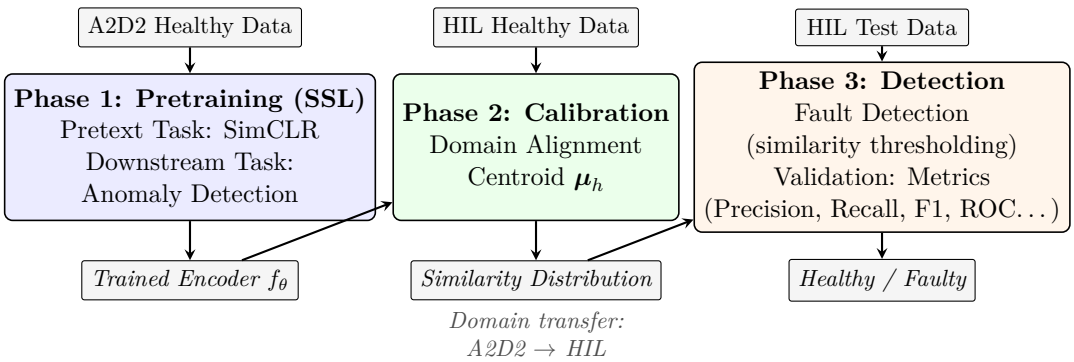

Figure 3.1: Overview of the three-phase fault detection methodology. Phase 1 trains the encoder on healthy real-world data using SimCLR (pretext task) and prepares it for the downstream anomaly detection task. Phase 2 calibrates the healthy reference centroid from a short HIL recording, thereby aligning the source and target domains. Phase 3 performs fault detection by comparing test embeddings to the centroid and validates the results using comprehensive metrics.

3.1 Problem Formulation

Let a multivariate automotive sensor signal be denoted as $\mathbf{x}(t) \in \mathbb{R}^C$, where $C = 2$ corresponds to the two selected channels: accelerator pedal position and vehicle speed. The continuous time series is segmented into fixed-length windows:

$$\mathbf{W}_k = \Big[\mathbf{x}(kS),\ \mathbf{x}(kS+1),\ \ldots,\ \mathbf{x}(kS+L-1)\Big] \in \mathbb{R}^{L\times C} \tag{3.1}$$

where L is the window length and S is the stride. Each window represents an independent observation of the system's operational state.

The objective is to learn an encoder function

$$f_\theta : \mathbb{R}^{L\times C} \to \mathbb{R}^d \tag{3.2}$$

that maps each window to a d-dimensional embedding space with the following properties:

1. **Compactness:** Windows originating from healthy operation (distribution p_{healthy}) should form a compact cluster in the embedding space.

2. **Separability:** Windows containing sensor faults (distribution p_{fault}) should produce embeddings that significantly deviate from this cluster.

Crucially, the encoder must be learned *without access to any labelled fault examples.* This constraint eliminates supervised and semi-supervised approaches and motivates the adoption of self-supervised contrastive learning. Furthermore, the encoder must generalise across domains—specifically from the real-world driving data used for training (A2D2) to the simulated hardware-in-the-loop data used for evaluation (HIL). This cross-domain transfer requirement precludes any domain-specific fine-tuning and demands that the learned representations be invariant to distributional shift between data acquisition systems.

3.2 Self-Supervised Contrastive Learning with SimCLR

Contrastive learning learns representations by enforcing invariance under data augmentations while ensuring discriminability between different inputs. Among the various contrastive frameworks, SimCLR (T. Chen et al., 2020) is selected for its conceptual simplicity, strong empirical performance, and straightforward adaptability to one-dimensional time series.

3.2.1 The Contrastive Learning Paradigm

Given an input window $\mathbf{W}$, two stochastic augmentation functions $\mathcal{T}_1, \mathcal{T}_2 \sim \mathcal{T}$ generate correlated views:

$$\tilde{\mathbf{W}}^{(1)} = \mathcal{T}_1(\mathbf{W}), \qquad \tilde{\mathbf{W}}^{(2)} = \mathcal{T}_2(\mathbf{W}) \tag{3.3}$$

The encoder f_θ and a projection head g_ϕ map these views to projections:

$$\mathbf{z}^{(1)} = g_\phi\Big(f_\theta(\tilde{\mathbf{W}}^{(1)})\Big), \qquad \mathbf{z}^{(2)} = g_\phi\Big(f_\theta(\tilde{\mathbf{W}}^{(2)})\Big) \tag{3.4}$$

The contrastive objective pulls the projections of positive pairs (views of the same original window) together while pushing projections of negative pairs (views of different windows) apart.

3.2.2 Data Augmentation Design

Data augmentation defines the invariances that the encoder must acquire. For automotive sensor signals, three augmentation families are designed to reflect benign variability that should not affect the representation of normal operation:

Gaussian Jittering

Minor random noise is always present in real sensor readings due to electromagnetic interference, quantisation error, and thermal effects. An encoder should be invariant to such benign fluctuations. Gaussian jittering adds independent Gaussian noise to each time step:

$$\tilde{x}^{(j)}[t] = x^{(j)}[t] + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma_j^2) \tag{3.5}$$

The noise scale σ_j is set relative to the normalised signal variance.

Amplitude Scaling

Manufacturing tolerances, temperature drift, or calibration differences can introduce slight multiplicative offsets. Amplitude scaling multiplies the entire window by a random scalar:

$$\tilde{x}^{(j)}[t] = s \cdot x^{(j)}[t], \quad s \sim \text{Uniform}(s_{\min}, s_{\max}) \tag{3.6}$$

This augmentation encourages invariance to small gain variations.

Temporal Masking

To prevent the encoder from relying on any single temporal segment, a contiguous block is masked (set to zero):

$$\tilde{x}^{(j)}[t] = \begin{cases} 0 & \text{if } t_0 \leq t < t_0 + m \\ x^{(j)}[t] & \text{otherwise} \end{cases} \tag{3.7}$$

where $t_0 \sim \text{Uniform}(0, L - m)$ and $m = \lfloor rL \rfloor$ with mask ratio r. This forces the encoder to infer the global structure from the remaining context, analogous to masked language modelling in NLP.

Each augmentation is applied stochastically with a fixed probability, ensuring that the two views of the same window undergo different augmentation combinations. This stochasticity increases the difficulty and effectiveness of the contrastive task.

3.2.3 Encoder Architecture

The encoder f_θ is a deep convolutional neural network operating on the temporal dimension. Convolutional architectures are well-suited to time series because they exploit local temporal correlations, are translation-invariant, and naturally support hierarchical feature extraction.

The design follows three architectural principles:

1. **Hierarchical abstraction:** Early layers capture short-term patterns (e.g., sudden accelerator release); deeper layers integrate information over longer temporal extents.
2. **Progressive channel expansion:** The number of feature maps increases with depth to compensate for reduced spatial resolution, a practice established in image-based CNNs.
3. **Non-linearity and normalisation:** Each convolutional block is followed by batch normalisation and a ReLU activation to accelerate training and improve gradient flow.

Formally, a convolutional block is defined as:

$$\mathbf{h}_{\ell+1} = \mathrm{MaxPool}\Big(\mathrm{ReLU}\Big(\mathrm{BN}\Big(\mathrm{Conv1d}_\ell(\mathbf{h}_\ell)\Big)\Big)\Big) \tag{3.8}$$

where $\mathbf{h}_0 = \tilde{\mathbf{W}}$ is the augmented input window.

The final embedding is obtained via global average pooling, which aggregates temporal information without learned parameters and produces a fixed-dimensional representation regardless of the exact window length:

$$\mathbf{h} = \mathrm{GlobalAvgPool1d}(\mathbf{h}_B) \in \mathbb{R}^d \tag{3.9}$$

3.2.4 Projection Head

The projection head g_ϕ is a small multi-layer perceptron that maps the embedding $\mathbf{h}$ to a lower-dimensional space $\mathbf{z} = g_\phi(\mathbf{h})$ where the contrastive loss is applied. T. Chen et al. (2020) demonstrate that applying the loss in the projection space—rather than directly on the embedding—substantially improves

representation quality by stripping away domain-specific information that is irrelevant for invariance. The projection head consists of two linear layers with a batch normalisation and ReLU between them:

$$g_\phi(\mathbf{h}) = \mathbf{W}^{(2)} \cdot \mathrm{ReLU}\Big(\mathrm{BN}(\mathbf{W}^{(1)}\mathbf{h} + \mathbf{b}^{(1)})\Big) + \mathbf{b}^{(2)} \tag{3.10}$$

The projection head is discarded after training; only the encoder f_θ is retained for downstream anomaly detection.

3.2.5 NT-Xent Loss

For a batch of N windows, $2N$ augmented views are generated, yielding $2N$ projections $\{\mathbf{z}_1, \mathbf{z}_2, \ldots, \mathbf{z}_{2N}\}$. For each positive pair $(\mathbf{z}_i, \mathbf{z}_j)$—the two views of the same original window—the remaining $2N - 2$ views act as negative examples.

The cosine similarity between two projections is defined as:

$$\mathrm{sim}(\mathbf{z}_i, \mathbf{z}_j) = \frac{\mathbf{z}_i^\top \mathbf{z}_j}{\|\mathbf{z}_i\| \|\mathbf{z}_j\|} \tag{3.11}$$

The loss for a positive pair (i, j) is:

$$\ell_{i,j} = -\log \frac{\exp\Big(\mathrm{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau\Big)}{\sum_{k=1}^{2N} \mathbf{1}_{[k \neq i]} \exp\Big(\mathrm{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau\Big)} \tag{3.12}$$

where τ is a temperature parameter that controls the sharpness of the distribution. A low temperature forces the model to focus on the hardest negatives, while a high temperature produces softer probability distributions. The total loss for the batch is the average over all positive pairs:

$$\mathcal{L} = \frac{1}{2N} \sum_{k=1}^{N} \Big[\ell_{2k-1,\,2k} + \ell_{2k,\,2k-1}\Big] \tag{3.13}$$

Minimising $\mathcal{L}$ maximises the agreement between positive pairs while minimising agreement between negative pairs, yielding representations that are both invariant to augmentations and discriminative across samples.

3.2.6 Optimisation Objective

The encoder f_θ and projection head g_ϕ are jointly optimised to minimise $\mathcal{L}$. The Adam optimiser (Kingma and Ba, 2015) is employed due to its adaptive per-parameter learning rates and robust performance across a wide range of deep learning tasks. Weight decay (L2 regularisation) is applied to both networks to prevent overfitting.

3.3 Healthy Reference Modeling

After contrastive pretraining, the projection head is discarded and the encoder is frozen (no further weight updates). A small set of healthy windows from the target domain—denoted $\mathcal{H} = \{\mathbf{W}_1^{\text{cal}}, \ldots, \mathbf{W}_M^{\text{cal}}\}$—is passed through the encoder to obtain reference embeddings $\{\mathbf{h}_1^{\text{cal}}, \ldots, \mathbf{h}_M^{\text{cal}}\}$.

The healthy centroid is defined as the arithmetic mean of these embeddings:

$$\boldsymbol{\mu}_h = \frac{1}{M} \sum_{i=1}^{M} \mathbf{h}_i^{\text{cal}} \tag{3.14}$$

The centroid represents the nominal operating state of the system in the embedding space. The choice of the mean is motivated by the expectation that healthy embeddings form a roughly isotropic cluster; alternative location estimators (e.g., median) could be considered if the cluster exhibits significant skewness, but the mean is both statistically efficient and computationally simple.

3.4 Similarity-Based Anomaly Detection

For any test window $\mathbf{W}$, its embedding $\mathbf{h} = f_\theta(\mathbf{W})$ is extracted, and the cosine similarity to the healthy centroid is computed:

$$s(\mathbf{W}) = \frac{\mathbf{h} \cdot \boldsymbol{\mu}_h}{\|\mathbf{h}\| \|\boldsymbol{\mu}_h\|} \tag{3.15}$$

Cosine similarity is preferred over Euclidean distance because it is invariant to the norm of the embedding, which can vary arbitrarily during contrastive

training. Values range from -1 (opposite direction) to $+1$ (identical direction); healthy samples should exhibit high positive similarity.

A window is classified as faulty if its similarity falls below a detection threshold δ:

$$\text{Pred}(\mathbf{W}) = \begin{cases} \text{Healthy} & \text{if } s(\mathbf{W}) \geq \delta \\ \text{Faulty} & \text{if } s(\mathbf{W}) < \delta \end{cases} \tag{3.16}$$

Rather than selecting a single threshold a priori, the proposed methodology evaluates multiple thresholds derived from the distribution of healthy similarity scores. Specifically, the percentiles of $\{s(\mathbf{W}_i^{\text{cal}})\}_{i=1}^{M}$ serve as candidate thresholds. This systematic exploration characterises the precision–recall trade-off and enables the selection of an operating point that satisfies safety-critical recall requirements.

3.5 Cross-Domain Transfer Setting

A central requirement of this thesis is that the encoder, trained exclusively on real driving data (A2D2), must perform fault detection on simulated hardware-in-the-loop data (HIL) without any retraining or fine-tuning. This constitutes a domain transfer problem: the source domain $\mathcal{D}_{\text{A2D2}}$ and target domain $\mathcal{D}_{\text{HIL}}$ exhibit different statistical distributions due to differences in sensor hardware, recording environment, and driving scenarios.

The proposed methodology addresses domain transfer through two design decisions:

1. **Consistent normalisation:** The Z-score normalisation parameters $\{\mu_j, \sigma_j\}$ are estimated exclusively from the source domain (A2D2) and applied unchanged to the target domain (HIL). This linear transformation preserves the relative scale and offset between the two domains, ensuring that the encoder, trained on source-normalised inputs, receives target-normalised inputs that are statistically comparable.

2. **Augmentation-induced invariance:** The contrastive augmentations (jittering, scaling, masking) are designed to be domain-agnostic. By

enforcing invariance to perturbations that are common across domains, the encoder learns representations that are robust to distributional shift.

This transfer setting is deliberately challenging: no HIL data is used during pretraining, and only a brief healthy recording from the HIL system is permitted for calibration. Successful detection under these constraints demonstrates the practical viability of the proposed framework for real-world deployment, where obtaining large labelled datasets from each target platform is infeasible.

3.6 Summary

This chapter has established the methodological foundation of the proposed self-supervised fault detection framework. The problem is formulated as learning a similarity-preserving embedding space without labelled faults. SimCLR is adopted as the contrastive learning paradigm and adapted for multivariate time series through domain-appropriate augmentations and a 1D convolutional encoder. The anomaly detection stage is simple yet theoretically grounded: a healthy centroid is estimated from a small calibration set, and cosine similarity to this centroid serves as the anomaly score. A range of thresholds is considered to fully characterise the precision–recall trade-off. Finally, evaluation metrics are defined, with emphasis on recall as the primary safety-critical criterion and on threshold-independent measures (ROC-AUC, PR-AP) for comprehensive assessment.

The next chapter details the concrete implementation of this methodology, including dataset processing, hyperparameter selection, training procedures, and the experimental setup used to obtain the results reported in Chapter 5.

4 Implementation and Experimental Setup

This chapter describes the concrete realization of the methodology presented in Chapter 3. The goal is to design a Self-Supervised Learning (SSL) pipeline that learns robust representations from real driving data (without labels), transfers knowledge to HIL (where faults are known), and enables anomaly detection across domains and scenarios. All implementation decisions—from dataset parsing to neural network training, calibration, and evaluation—are documented with sufficient detail to ensure reproducibility. The chapter follows the three-phase pipeline introduced in Section 3.1 and is structured accordingly: data acquisition and preprocessing (Phase 0), SimCLR training implementation (Phase 1), calibration of the healthy reference (Phase 2), and the fault detection evaluation protocol (Phase 3). Intermediate results are presented at each stage to validate the correct operation of the implemented components.

4.1 Datasets

Two distinct data sources are employed in this thesis, reflecting the cross-domain transfer scenario from real-world driving to hardware-in-the-loop simulation.

4.1.1 A2D2 Training Data

The Audi Autonomous Driving Dataset (A2D2) (Audi AG, 2020) provides the training data for self-supervised representation learning. The dataset contains timestamped sensor recordings from real test drives in Germany. Only the `bus_signals.json` files are used, as they contain the controller area network (CAN) bus signals relevant to this work.

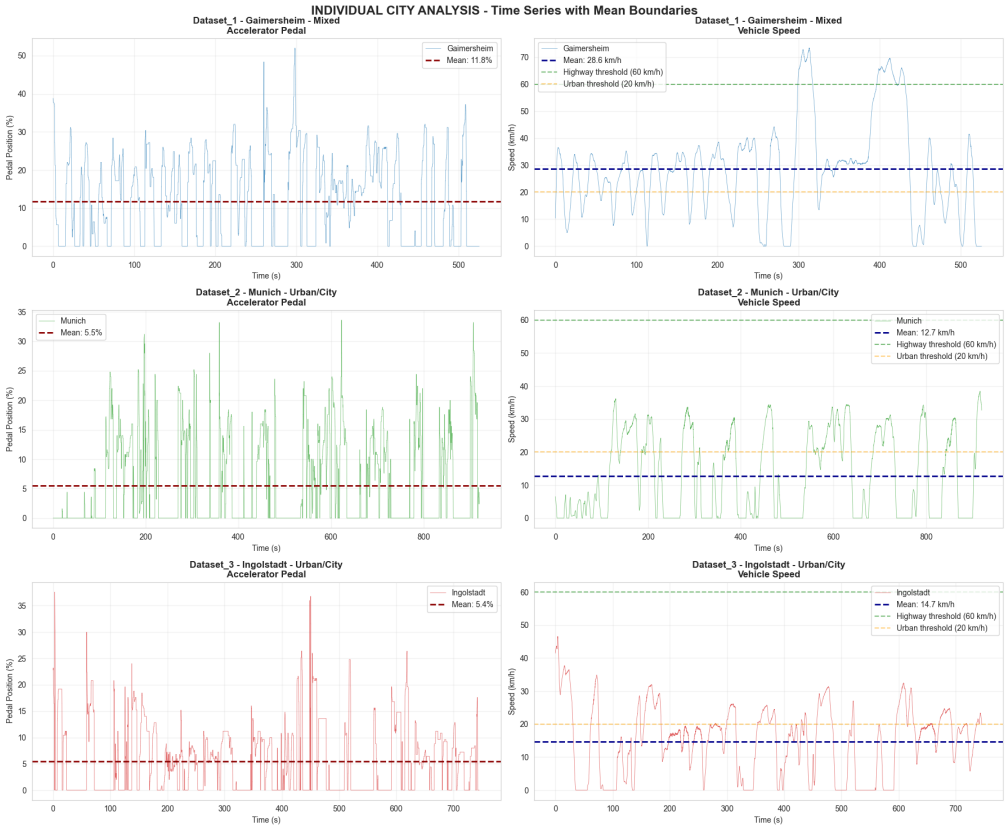

Figure 4.1: Visualization of the three A2D2 driving scenarios: Gaimersheim (mixed), Munich (urban), and Ingolstadt (urban). The plots show speed profiles and driving characteristics for each dataset, illustrating the diversity of real-world conditions used for training.

File selection. Three JSON files are selected based on file size (greater than 1 MB to exclude header-only files) and recording duration. These files represent diverse driving conditions (urban, extra-urban) and are listed in Table 4.1, which now includes the corresponding city and road type for each recording.

Table 4.1: A2D2 JSON files used for training, with city and road type classification.

Filename	City	Road Type	Original samples (acc)	Original samples (speed)	Duration (s) at 100 Hz
20190401121727_bus_signals.json	Munich	Urban/City	91,968	45,984	919.7
20190401145936_bus_signals.json	Ingolstadt	Urban/City	74,520	37,260	745.2
20180810150607_bus_signals.json	Gaimersheim	Mixed	52,576	26,288	525.8
Total			**219,064**	**109,532**	**2190.6**

JSON parsing. Each file is loaded and parsed using Python's `json` module. The top-level keys `accelerator_pedal` and `vehicle_speed` are located; under each, the `values` field contains a list of [timestamp$_{\mu s}$, value] pairs. Timestamps are converted from microseconds to seconds and normalized such that the first timestamp of each file is set to zero.

Sampling rate verification. The median sampling interval is computed from consecutive timestamps. The accelerator pedal signal exhibits a median interval of 0.0100 s (100 Hz), while the vehicle speed signal shows a median interval of 0.0200 s (50 Hz). This discrepancy is addressed by upsampling.

Speed upsampling. The vehicle speed signal is linearly interpolated to 100 Hz using `scipy.interpolate.interp1d` with `kind='linear'`. Linear interpolation is physically justified: vehicle speed changes smoothly due to inertia, and the interpolation interval (0.01 s) is an order of magnitude smaller than typical acceleration and deceleration events.

Concatenation. The processed data from the three files are concatenated into a single pandas DataFrame with columns `accelerator`, `speed`, and `time_rel`. The final training set contains 219,064 samples at 100 Hz, corresponding to 36.5 minutes of driving.

4.1.2 HIL Calibration and Test Data

The HIL dataset originates from a dSPACE hardware-in-the-loop test bench at TU Clausthal (Abboush et al., 2022). It comprises one healthy recording and six fault-injected recordings, each stored as a CSV file exported from the dSPACE

Measurement Data Format (MDF). The dataset simulates a gasoline engine model with realistic sensor dynamics and enables controlled fault injection.

CSV parsing. The CSV files follow a non-standard format with metadata headers. The parsing algorithm proceeds as follows:

1. Read the file line by line until a line beginning with `"path,"` is encountered. This line contains the column headers.

2. Split the header line by commas; identify the columns corresponding to accelerator pedal position and vehicle speed via keyword matching (`"AccPedal"` or `"accelerator"` for accelerator; `"v_Vehicle"` or `"speed"` for speed).

3. Continue reading lines until a line beginning with `"trace_values,"` is found. Subsequent lines contain the numerical data, with values separated by commas.

4. Parse the numerical data lines and construct a DataFrame with columns renamed to the canonical names `accelerator` and `speed`.

5. Handle missing values (encoded as `NaN` in the export) via linear interpolation.

Sampling rate. All HIL recordings are already sampled at 100 Hz, verified by computing the median timestamp difference from a time column when present. No upsampling is therefore required.

Table 4.2 lists the seven HIL files with their properties.

Table 4.2: HIL data files used for calibration and testing.

Filename	**Purpose**	**Faulty sensor**	**Fault type**	**Samples**
healthy.csv	Calibration (30 %) + Test (70 %)	–	–	31,881
acc_fault_gain.csv	Test	Accelerator	Gain (1.5×)	31,546
acc_fault_noise.csv	Test	Accelerator	Gaussian noise	32,205
acc_fault_stuck.csv	Test	Accelerator	Stuck at 0 %	31,684
rpm_fault_gain.csv	Test	Speed	Gain (1.2×)	32,276
rpm_fault_noise.csv	Test	Speed	Gaussian noise	31,647
rpm_fault_stuck.csv	Test	Speed	Stuck at 0 km/h	32,653

4.2 Data Preprocessing Pipeline

All preprocessing steps are implemented in Python using `numpy`, `pandas`, and `scikit-learn`. The pipeline is designed to be deterministic and reproducible, with random seeds fixed where applicable.

4.2.1 Normalization

Z-score normalization is applied independently to each sensor channel. The mean μ_j and standard deviation σ_j are computed exclusively from the A2D2 training data:

$$\mu_j = \frac{1}{T}\sum_{t=1}^{T} x^{(j)}[t], \qquad \sigma_j = \sqrt{\frac{1}{T}\sum_{t=1}^{T}\left(x^{(j)}[t] - \mu_j\right)^2} \tag{4.1}$$

These parameters are saved using `joblib` and subsequently applied to both the A2D2 data (for training) and all HIL data (for calibration and testing) without recomputation. This ensures that the same linear transformation is applied to both domains, a critical requirement for cross-domain transfer.

4.2.2 Sliding Window Segmentation

The normalized time series are segmented into overlapping windows using a custom PyTorch `Dataset` class, `TimeSeriesWindowDataset`. The class precomputes all start indices and performs on-the-fly window extraction and tensor conversion.

Window parameters.

- Window length $L = 200$ samples (2 seconds at 100 Hz)
- Stride $S = 100$ samples (50 % overlap)

These parameters are chosen to capture meaningful driving dynamics (acceleration, braking, gear changes) while providing sufficient training examples via overlap. For the A2D2 training set of 219,064 samples, this yields $K = \lfloor (219064 - 200)/100 \rfloor + 1 = 2189$ windows.

Output format. Each window is returned as a `torch.FloatTensor` of shape $(C, L) = (2, 200)$, following the PyTorch convention for 1D convolutions (channels first). This format is directly consumable by the encoder.

4.2.3 Dataset Fusion Implementation

The fusion of A2D2 and HIL data into a unified processing pipeline is achieved through the following alignment steps:

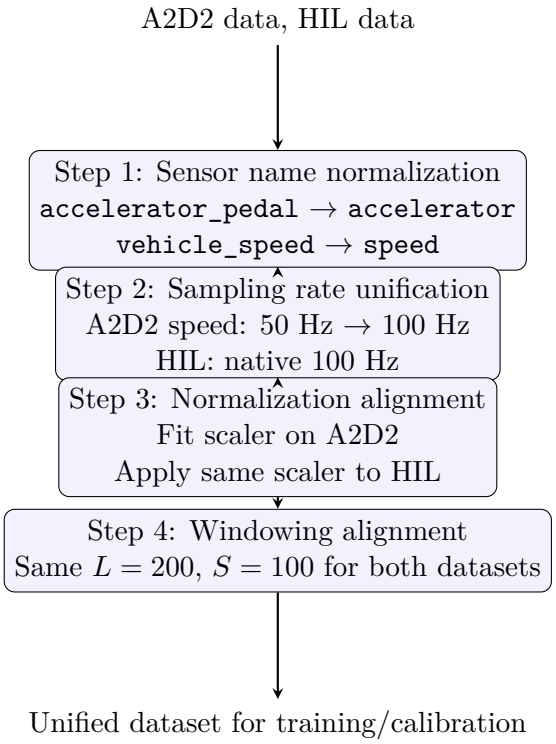

Figure 4.2: Dataset fusion pipeline: alignment of A2D2 and HIL data through sensor name normalization, sampling rate unification, normalization alignment, and consistent windowing.

1. **Sensor name normalization:** A2D2 columns `accelerator_pedal` and `vehicle_speed` are renamed to `accelerator` and `speed`. HIL columns are mapped to these canonical names during CSV parsing.

2. **Sampling rate unification:** A2D2 speed is upsampled to 100 Hz; HIL data is native 100 Hz.

3. **Normalization alignment:** The same `StandardScaler` fitted on A2D2 is applied to all HIL data.

4. **Windowing alignment:** The same `TimeSeriesWindowDataset` with identical L and S is used for both datasets.

4.3 SimCLR Training Implementation

4.3.1 Data Augmentation Modules

Three augmentation classes are implemented as callable PyTorch modules inheriting from `nn.Module`. Each augmentation is applied stochastically with probability $p = 0.5$ per view.

GaussianJitter. Adds independent Gaussian noise with standard deviation `scale`. The noise is sampled per time step and per channel. Implementation:

```
class GaussianJitter(nn.Module):
    def __init__(self, scale=0.1):
        super().__init__()
        self.scale = scale
    def forward(self, x):
        if self.training and torch.rand(1) < 0.5:
            noise = torch.randn_like(x) * self.scale
            return x + noise
        return x
```

AmplitudeScaling. Multiplies the entire window by a scalar sampled uniformly from `scale_range`.

TemporalMask. Masks a contiguous block of length $m = \lfloor \text{mask_ratio} \cdot L \rfloor$. The start index is sampled uniformly from $[0, L - m]$.

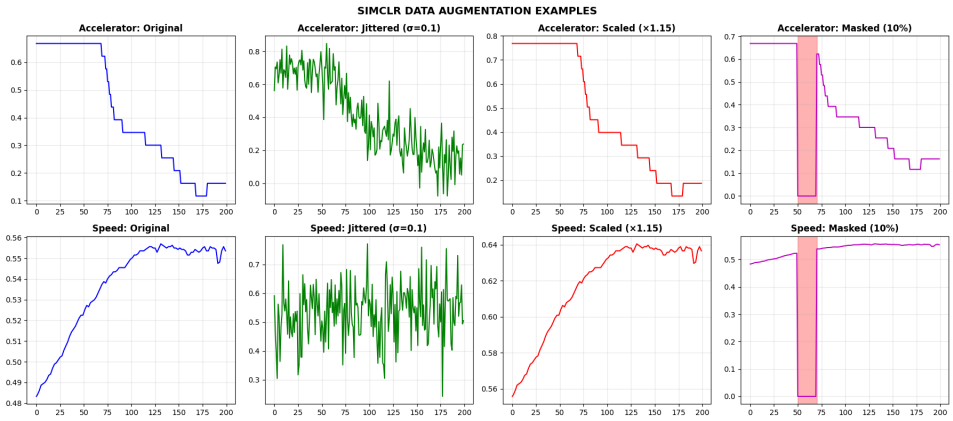

Figure 4.3: Examples of the three implemented data augmentations.

4.3.2 Encoder Architecture Implementation

The encoder is implemented as `TimeSeriesSimCLREncoder`, a subclass of `nn.Module`. Table 4.3 provides the complete layer-by-layer specification.

Table 4.3: Layer-by-layer implementation of the 1D-CNN encoder.

Block	Layer	In	Out	Kernel	Stride	Output shape
–	Input	–	–	–	–	$(B, 2, 200)$
1	Conv1d	2	64	7	2	$(B, 64, 100)$
	BatchNorm1d	–	–	–	–	$(B, 64, 100)$
	ReLU + MaxPool1d	–	–	pool=2	2	$(B, 64, 50)$
2	Conv1d	64	128	5	2	$(B, 128, 25)$
	BatchNorm1d	–	–	–	–	$(B, 128, 25)$
	ReLU + MaxPool1d	–	–	pool=2	2	$(B, 128, 12)$
3	Conv1d	128	256	3	1	$(B, 256, 12)$
	BatchNorm1d	–	–	–	–	$(B, 256, 12)$
	ReLU + MaxPool1d	–	–	pool=2	2	$(B, 256, 6)$
–	AdaptiveAvgPool1d	–	–	output=1	–	$(B, 256, 1)$
–	Flatten	–	–	–	–	$(B, 256)$

4.3.3 Projection Head Implementation

The projection head is implemented as:

```
class ProjectionHead(nn.Module):
    def __init__(self, input_dim=256, hidden_dim=256, output_dim=128):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.bn1 = nn.BatchNorm1d(hidden_dim)
        self.relu = nn.ReLU(inplace=True)
        self.fc2 = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        x = self.fc1(x)
        x = self.bn1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x
```

No non-linearity is applied after the final linear layer, following the original SimCLR design.

4.3.4 Hyperparameter Configuration

Table 4.4 lists the final hyperparameters selected after preliminary experimentation. All values are fixed for the main training run.

Table 4.4: Final hyperparameter configuration for SimCLR training.

Hyperparameter	**Value**	**Notes**
Window length L	200	2 seconds at 100 Hz
Stride S	100	50 % overlap
Batch size N	128	GPU memory constraint
Epochs	50	Convergence observed by epoch 40
Optimizer	Adam	(Kingma and Ba, 2015)
Learning rate	1×10^{-3}	Fixed, no decay
Weight decay	1×10^{-4}	L2 regularization
Temperature τ	0.5	As per T. Chen et al. (2020)
Jitter scale	0.1	After normalization
Scale range	$[0.8, 1.2]$	$\pm 20\,\%$
Mask ratio	0.1	20 samples
Augmentation probability	0.5	Per view, per augmentation
Embedding dimension d	256	Encoder output
Projection dimension p	128	NT-Xent loss space

4.3.5 Training Loop Implementation

The training loop is implemented in PyTorch with the following structure:

1. **Initialization:** Encoder and projection head are instantiated and moved to GPU (CUDA) if available. Adam optimizer is initialized with the specified learning rate and weight decay.

2. **Data loading:** A `DataLoader` wraps the `TimeSeriesWindowDataset` with batch size $N = 128$, shuffling enabled, and `drop_last=True` to ensure consistent batch size.

3. **Per-batch processing:**

   - A batch of windows $\mathbf{W}_b$ of shape $(N, 2, 200)$ is loaded.

   - Two sets of stochastic augmentations are applied independently to the batch, yielding $\tilde{\mathbf{W}}_b^{(1)}$ and $\tilde{\mathbf{W}}_b^{(2)}$.

- Both views are passed through the encoder → projection head to obtain projections $\mathbf{z}_b^{(1)}, \mathbf{z}_b^{(2)} \in \mathbb{R}^{N \times 128}$.
- The NT-Xent loss is computed using a numerically stable implementation that avoids explicit construction of the full $2N \times 2N$ similarity matrix.
- Loss is backpropagated; optimizer step updates both encoder and projection head parameters.

4. **Logging and checkpointing:** Loss values are recorded per step and per epoch. Model checkpoints (encoder weights, optimizer state) are saved every 10 epochs.

4.4 Calibration Implementation

Following training, the projection head is discarded and the encoder is switched to evaluation mode (`model.eval()`). The calibration phase uses the healthy HIL recording to establish the reference embedding distribution.

Calibration data selection. The healthy recording (`healthy.csv`, 31,881 samples) is split chronologically: the first 30 % (9,564 samples) is reserved for calibration, while the remaining 70 % (22,317 samples) is held out as unseen healthy test data for the binary classification evaluation. This split ensures that the binary evaluation includes genuinely unseen healthy windows alongside the fault windows.

Window extraction. The calibration segment is normalized using the saved A2D2 scaler and passed to `TimeSeriesWindowDataset` with $L = 200$, $S = 100$, yielding $M = 94$ calibration windows. The held-out healthy test segment is processed identically, producing 222 test windows.

Embedding extraction. Each calibration window is passed through the encoder without gradient computation (`torch.no_grad()`). The resulting embeddings $\{\mathbf{h}_i^{\text{cal}}\}_{i=1}^{M}$ are stacked into a matrix $\mathbf{H} \in \mathbb{R}^{M \times 256}$.

Centroid computation. The healthy centroid is computed as the column-wise mean:

$$\boldsymbol{\mu}_h = \frac{1}{M} \sum_{i=1}^{M} \mathbf{h}_i^{\text{cal}} \in \mathbb{R}^{256} \tag{4.2}$$

Similarity distribution. Cosine similarity between each $\mathbf{h}_i^{\text{cal}}$ and $\boldsymbol{\mu}_h$ is computed. Figure 4.4 displays the histogram of these similarities.

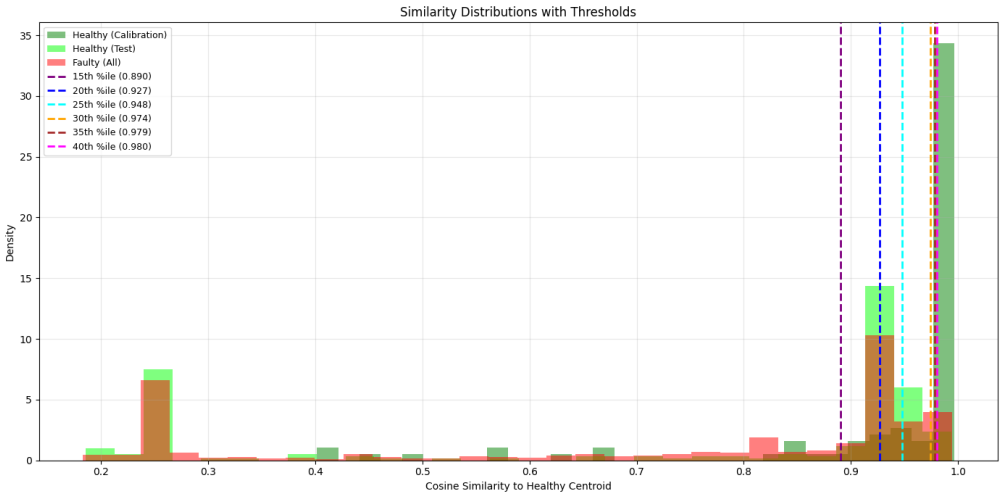

Figure 4.4: Healthy similarity distribution and selected thresholds.

Threshold selection. The 15th, 20th, 25th, 30th, 35th, and 40th percentiles of the healthy similarity distribution are computed. These six thresholds are saved for subsequent fault detection evaluation.

4.5 Fault Detection Evaluation Protocol

The evaluation protocol is designed to provide a comprehensive assessment of the detection framework under multiple operating points and across all fault types.

4.5.1 Per-Fault Evaluation

Each of the six HIL fault files is processed independently through the following procedure:

1. Load the CSV file and normalize using the saved A2D2 scaler.
2. Create a `TimeSeriesWindowDataset` with $L = 200$, $S = 100$.

3. Extract embeddings for all windows using the frozen encoder (no gradient).

4. Compute cosine similarity between each embedding and the healthy centroid $\boldsymbol{\mu}_h$.

5. For each of the six thresholds:

   - Classify windows as faulty if similarity $<$ threshold.

   - Since the entire file is known to contain a fault, assign ground truth label $y = 1$ (faulty) to all windows.

   - Compute confusion matrix entries (TP, FN) and derived metrics (recall, F1).

4.5.2 Binary Classification Evaluation

The binary evaluation combines all healthy windows—94 calibration windows and 222 held-out test windows (316 total, label $y = 0$)—with all fault windows from all six fault files (1,911 windows, label $y = 1$) into a unified test set. The same similarity-based detection is applied across the combined dataset, and metrics (precision, recall, F1, accuracy, confusion matrix) are computed for each threshold.

4.5.3 ROC and Precision-Recall Curves

ROC and Precision-Recall curves are generated by sweeping the detection threshold δ continuously from the minimum to the maximum observed similarity value across the combined test set. The ROC curve plots the true positive rate (recall) against the false positive rate; the AUC is computed via numerical integration. The Precision-Recall curve plots precision against recall; average precision (AP) is computed as the weighted mean of precisions at each threshold.

4.5.4 Reproducibility Measures

To ensure deterministic and reproducible results, the following measures are implemented:

1. **Random seed fixation:**

```
import random, numpy, torch
random.seed(42)
numpy.random.seed(42)
torch.manual_seed(42)
torch.cuda.manual_seed_all(42)
```

2. **CUDA determinism:**

```
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

3. **Saved artifacts:** The following objects are saved to disk using `joblib` and `torch.save`:
   - Fitted `StandardScaler` (A2D2 normalization parameters)
   - Trained encoder weights (final epoch)
   - Healthy centroid $\boldsymbol{\mu}_h$
   - Six percentile thresholds
   - Full training log (loss per step, per epoch)

4.6 Summary

This chapter has provided a complete, reproducible account of the implementation of the proposed self-supervised fault detection framework. The A2D2 and HIL datasets were processed through a unified preprocessing pipeline with consistent normalization and windowing. A 1D-CNN encoder and projection head were implemented in PyTorch and trained using the SimCLR contrastive objective with domain-appropriate augmentations. The calibration phase established a healthy centroid and six percentile thresholds using a brief HIL healthy

recording. The evaluation protocol was defined for both per-fault and binary classification, including ROC and Precision-Recall analysis. Computational environment specifications and reproducibility measures were documented to facilitate independent verification.

The following chapter presents the experimental results obtained by executing this implementation, including quantitative detection metrics, threshold analysis, and embedding visualizations.

5 Results and Discussion

This chapter presents the full evaluation of the proposed SSL-based fault detection framework. The results are organized in four parts: (1) training outcomes, (2) multi-threshold fault detection performance, (3) binary classification results (healthy vs. faulty), and (4) computing cost analysis.

5.1 Training Results

5.1.1 SimCLR Training Summary

Table 5.1 summarizes the SimCLR pretraining outcomes. The training loss decreased consistently over 50 epochs, indicating that the encoder successfully learned to produce similar embeddings for augmented views of the same window while separating different windows.

Table 5.1: SimCLR training summary.

Metric	**Value**
Training windows	2,189
Batches per epoch	17
Total training steps	850
Initial loss (epoch 1)	4.0927
Final loss (epoch 50)	3.7938
Loss reduction	7.3 %
Training time	95.46 s (1.59 min)
Encoder parameters	141,504
Projection head parameters	99,200
Total parameters	240,704

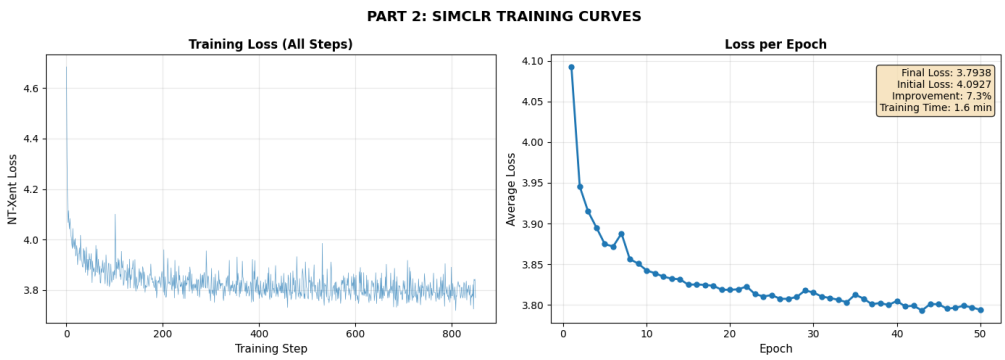

Figure 5.1: SimCLR training loss over 50 epochs. Left: per-step loss. Right: per-epoch average loss with standard deviation.

The loss curve (Figure 5.1) shows three characteristic phases: a rapid decrease during the first 10 epochs as the encoder learns basic signal structure, a slower decrease from epochs 10–30 as the representations are refined, and near-convergence from epochs 30–50 where further improvement is marginal. This behavior is consistent with the training dynamics reported by T. Chen et al. (2020) and confirms that 50 epochs are sufficient for convergence on this dataset.

5.2 Multi-Threshold Fault Detection Results

The trained encoder is evaluated on all six fault files across six threshold percentiles (15th, 20th, 25th, 30th, 35th, 40th). Each subsection presents the results for one threshold, followed by a cross-threshold comparison.

5.2.1 Results at 15th Percentile Threshold

Table 5.2: Fault detection results at the 15th percentile threshold ($\tau = 0.8901$).

Fault	**Windows**	**Detected**	**Recall**	**F1**
Acc gain	314	124	0.3949	0.5662
Acc noise	321	143	0.4455	0.6164
Acc stuck	315	128	0.4063	0.5779
Speed gain	321	245	0.7632	0.8657
Speed noise	315	154	0.4889	0.6567
Speed stuck	325	147	0.4523	0.6229
Average	–	–	**0.4919**	**0.6510**

At the 15th percentile, the threshold is highly conservative ($\tau = 0.8901$). Only windows with very low similarity to the healthy centroid are flagged, missing approximately half of the faulty windows. Speed gain faults are already detected at 76.3% recall, while accelerator faults remain below 45%.

5.2.2 Results at 20th Percentile Threshold

Table 5.3: Fault detection results at the 20th percentile threshold ($\tau = 0.9268$).

Fault	**Windows**	**Detected**	**Recall**	**F1**
Acc gain	314	158	0.5032	0.6695
Acc noise	321	195	0.6075	0.7558
Acc stuck	315	182	0.5778	0.7324
Speed gain	321	277	0.8629	0.9264
Speed noise	315	215	0.6825	0.8113
Speed stuck	325	189	0.5815	0.7354
Average	–	–	**0.6359**	**0.7718**

5.2.3 Results at 25th Percentile Threshold

Table 5.4: Fault detection results at the 25th percentile threshold ($\tau = 0.9477$).

Fault	**Windows**	**Detected**	**Recall**	**F1**
Acc gain	314	259	0.8248	0.9040
Acc noise	321	272	0.8474	0.9174
Acc stuck	315	258	0.8190	0.9005
Speed gain	321	309	0.9626	0.9810
Speed noise	315	272	0.8635	0.9267
Speed stuck	325	277	0.8523	0.9203
Average	–	–	**0.8616**	**0.9250**

5.2.4 Results at 30th Percentile Threshold

Table 5.5: Fault detection results at the 30th percentile threshold ($\tau = 0.9742$).

Fault	**Windows**	**Detected**	**Recall**	**F1**
Acc gain	314	278	0.8854	0.9392
Acc noise	321	282	0.8785	0.9353
Acc stuck	315	272	0.8635	0.9267
Speed gain	321	318	0.9907	0.9953
Speed noise	315	295	0.9365	0.9672
Speed stuck	325	287	0.8831	0.9379
Average	–	–	**0.9063**	**0.9503**

5.2.5 Results at 35th Percentile Threshold

Table 5.6: Fault detection results at the 35th percentile threshold ($\tau = 0.9786$).

Fault	**Windows**	**Detected**	**Recall**	**F1**
Acc gain	314	278	0.8854	0.9392
Acc noise	321	286	0.8910	0.9423
Acc stuck	315	274	0.8698	0.9304
Speed gain	321	320	0.9969	0.9984
Speed noise	315	302	0.9587	0.9789
Speed stuck	325	289	0.8892	0.9414
Average	–	–	**0.9152**	**0.9551**

5.2.6 Results at 40th Percentile Threshold

Table 5.7: Fault detection results at the 40th percentile threshold ($\tau = 0.9803$).

Fault	**Windows**	**Detected**	**Recall**	**F1**
Acc gain	314	279	0.8885	0.9410
Acc noise	321	286	0.8910	0.9423
Acc stuck	315	275	0.8730	0.9322
Speed gain	321	320	0.9969	0.9984
Speed noise	315	304	0.9651	0.9822
Speed stuck	325	292	0.8985	0.9465
Average	–	–	**0.9188**	**0.9571**

5.2.7 Cross-Threshold Comparison

Table 5.8 consolidates the overall detection performance across all six thresholds, enabling direct comparison.

Table 5.8: Cross-threshold comparison of overall detection performance.

Percentile	**Threshold**	**Avg Recall**	**Avg F1**
15th	0.8901	0.4919	0.6510
20th	0.9268	0.6359	0.7718
25th	0.9477	0.8616	0.9250
30th	0.9742	0.9063	0.9503
35th	0.9786	0.9152	0.9551
40th	0.9803	0.9188	0.9571

Note: The aggregate ROC-AUC for the similarity-based classifier across all possible thresholds is 0.6314.

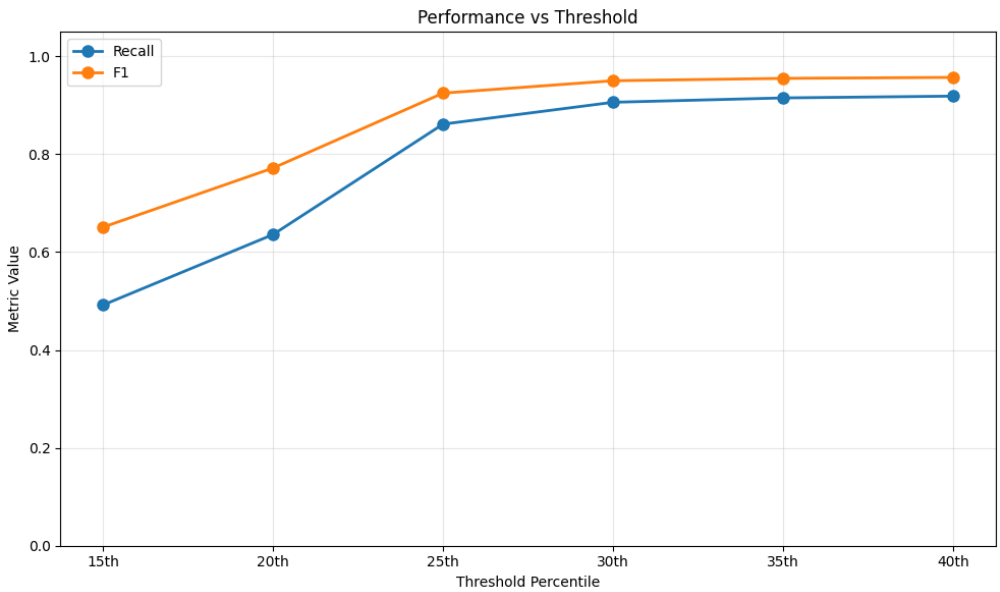

Figure 5.2: Detection metrics as a function of threshold percentile. Higher percentiles increase recall and F1.

The cross-threshold comparison reveals that recall increases monotonically from 49.2% at the 15th percentile to 91.9% at the 40th percentile, with the steepest improvement between the 20th and 25th percentiles. The F1-score increases from 0.6510 to 0.9571 across the threshold range. The aggregate ROC-AUC for the similarity-based classifier across all possible thresholds is 0.6314.

5.2.8 Per-Fault-Type Recall Heatmap

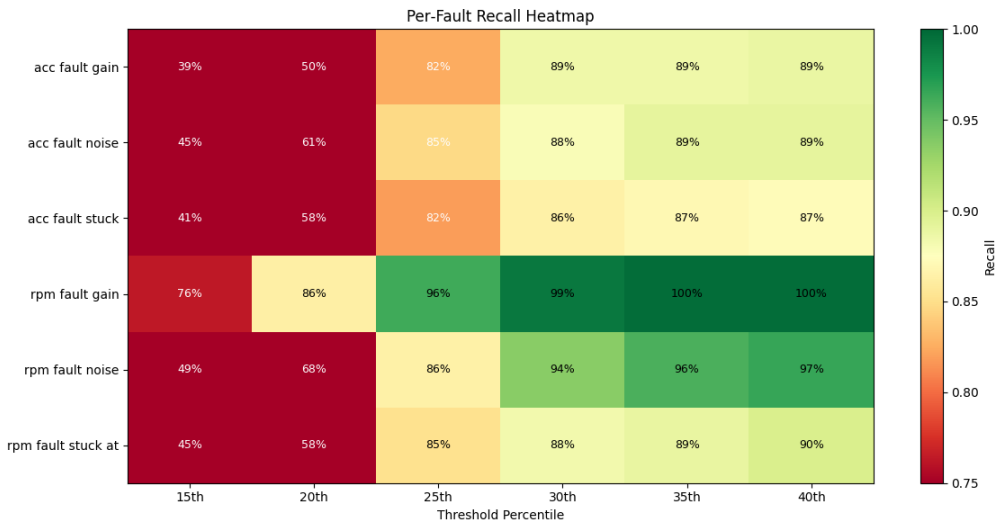

Figure 5.3: Per-fault recall heatmap across all six thresholds. Darker green indicates higher recall.

The recall heatmap (Figure 5.3) reveals characteristic differences between fault types and sensor locations:

- **Speed sensor faults** (bottom three rows) consistently achieve higher recall than accelerator sensor faults. At the 40th percentile, speed gain faults reach **99.7%** recall, and speed noise faults reach 96.5%. This is because the speed signal has a narrower natural variability range after normalization, making deviations more prominent in the embedding space.

- **Accelerator stuck-at faults** are the hardest to detect across all thresholds, achieving 40.6% at the 15th percentile and 87.3% at the 40th percentile. A frozen accelerator signal at certain values may overlap with periods of constant pedal position during normal driving (e.g., cruise or idle).

- **Speed gain faults** are the easiest to detect, reaching near-perfect recall (99.1–99.7%) from the 30th percentile onward. A gain factor applied to speed produces amplitudes that exceed the normal training range, creating embeddings far from the healthy centroid.

5.3 Binary Classification: Healthy vs. Faulty

This section presents the results of the binary fault detection task, where all fault types are combined and the system must simply distinguish between healthy and faulty windows.

5.3.1 Binary Confusion Matrices

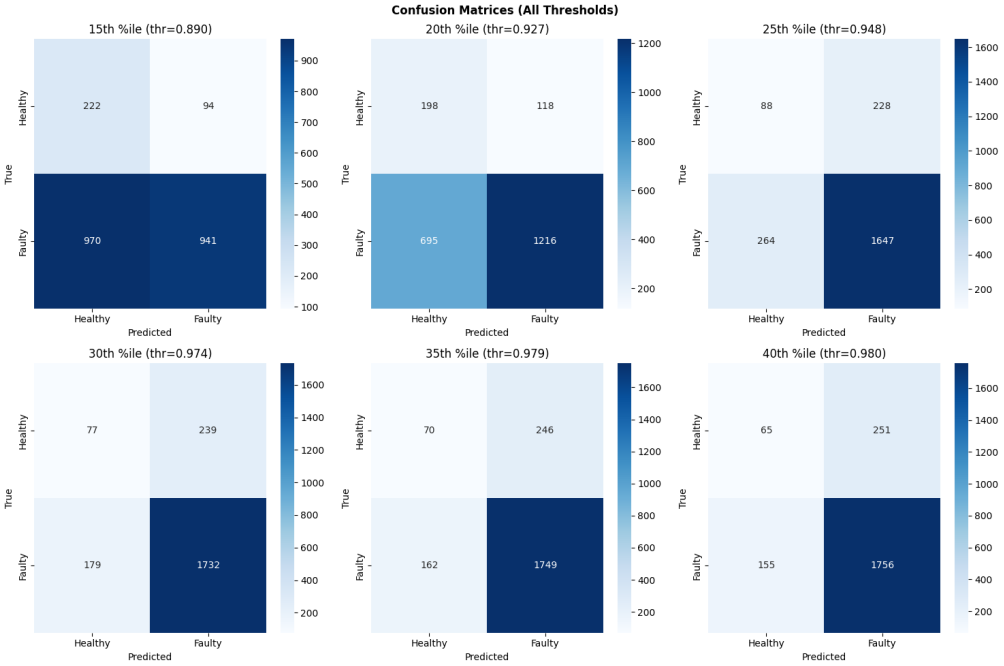

Figure 5.4: Binary classification confusion matrices for all six threshold percentiles. Each matrix shows the counts of true negatives (TN), false positives (FP), false negatives (FN), and true positives (TP).

5.3.2 Binary Classification Metrics

Table 5.9 reports precision, recall, F1-score, and accuracy for the binary classification task across all thresholds. The binary evaluation uses 316 healthy windows (94 calibration and 222 held-out test windows, label 0) and 1,911 fault windows from all six fault files combined (label 1).

Table 5.9: Binary classification results (healthy vs. faulty) across all thresholds.

Pctl.	Prec.	Recall	F1	Acc.	TN	FP	FN	TP
15th	0.9092	0.4924	0.6388	0.5222	222	94	970	941
20th	0.9115	0.6363	0.7495	0.6349	198	118	695	1,216
25th	0.8784	0.8619	0.8700	0.7791	88	228	264	1,647
30th	0.8787	0.9063	0.8923	0.8123	77	239	179	1,732
35th	0.8767	0.9152	0.8955	0.8168	70	246	162	1,749
40th	0.8749	0.9189	0.8964	0.8177	65	251	155	1,756

The binary results show that recall increases from 49.2% to 91.9% as the threshold percentile rises, while precision decreases from 90.9% to 87.5%. At the 40th percentile, the system correctly identifies 1,756 of 1,911 fault windows. The confusion matrices (Figure 5.4) visually demonstrate this trade-off: as the threshold percentile increases, TP counts grow (more faults detected) while FP also increases as the threshold encroaches on the healthy similarity distribution.

5.3.3 ROC Curves

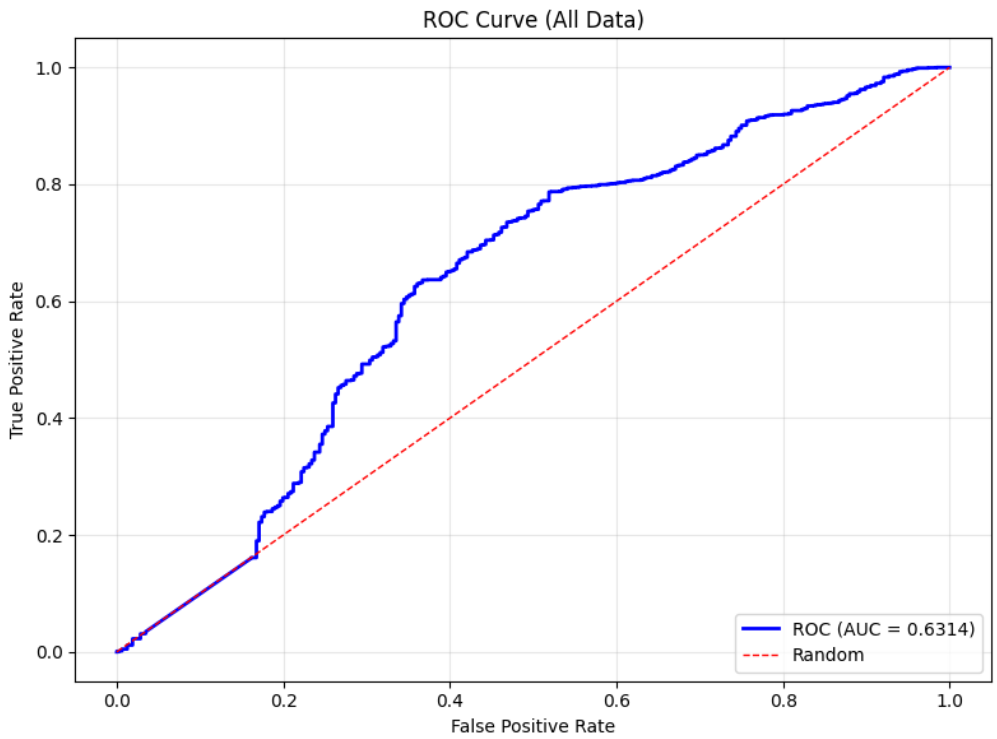

Figure 5.5: ROC curve for binary fault detection. The AUC is 0.6314. The diagonal dashed line represents a random classifier.

The ROC-AUC of 0.6314 indicates that the learned representations provide discriminative power above the 0.5 baseline of a random classifier, though there is considerable overlap between healthy and faulty similarity distributions. This overlap arises from the cross-domain transfer setting, where the encoder is trained on A2D2 data but evaluated on HIL data with different statistical characteristics.

5.4 Similarity Distribution Analysis

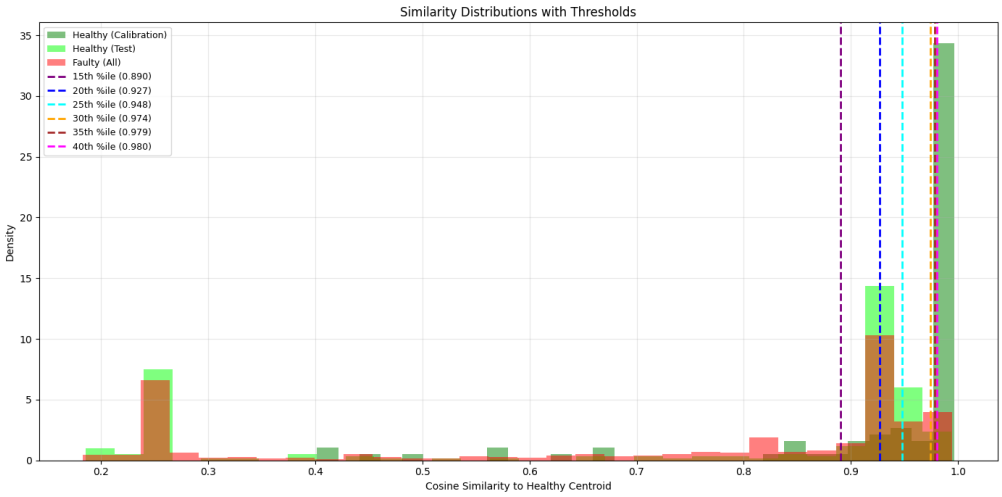

Figure 5.6: Cosine similarity distributions for healthy calibration (green), healthy test (lime), and faulty (red) windows, with vertical dashed lines marking all six threshold percentiles.

The similarity distribution plot (Figure 5.6) provides visual insight into the separation between healthy and faulty data in the embedding space. The healthy calibration distribution is concentrated at high similarity values (close to 1.0), reflecting the consistency of the learned healthy representations. The healthy test distribution shows a broader spread, indicating natural variability across the full healthy recording. The faulty distribution is shifted toward lower similarity values, though with overlap in the high-similarity region. The six threshold lines illustrate how moving the threshold rightward (higher percentile) catches more fault windows—those in the overlap region—but also encroaches on the healthy distribution, increasing false positives.

5.5 Computing Cost Analysis

Table 5.10 presents the measured computing times for each pipeline stage, addressing the supervisor's requirement to discuss computational costs in relation to training and testing time.

Table 5.10: Computing cost analysis for all pipeline stages (CPU execution).

Phase	Operation	Time (s)	Time (min)
Part 1	A2D2 data loading & preprocessing	26.72	0.45
	Visualization	(incl. above)	–
Part 2	SimCLR training (50 epochs)	95.46	1.59
	Augmentation visualization	(incl. below)	–
	Total Part 2 (incl. visualization)	97.76	1.63
Part 3	Embedding extraction (all files)	2.75	0.05
	Threshold evaluation (6 thresholds)	0.32	<0.01
	Visualization and saving	6.96	0.12
Total	**End-to-end pipeline**	**134.51**	**2.24**

5.5.1 Discussion of Computing Costs

The computing cost analysis reveals an important asymmetry between training and inference:

- **Training is a one-time cost.** SimCLR training constitutes the majority of the total pipeline time (95.46 s, approximately 71% of the total). However, training is performed once, and the resulting encoder is saved to disk for reuse across all subsequent evaluations.

- **Inference is fast.** Embedding extraction for all 1,911 fault windows and 316 healthy windows takes only 2.75 s on CPU, corresponding to approximately 1.2 ms per window. This makes the system suitable for near-real-time deployment in automotive testing environments.

- **Calibration is lightweight.** Computing the healthy centroid and similarity distribution from 94 calibration windows is included in the embedding extraction time and takes a negligible fraction of a second.

- **Multi-threshold evaluation adds negligible overhead.** Evaluating all six thresholds takes only 0.32 s, as it requires only simple comparison operations on pre-computed similarity scores.

- **Total pipeline under 2.5 minutes.** The entire end-to-end pipeline—from raw data loading through training to full evaluation—completes in under 135 seconds on CPU. This demonstrates the practical efficiency of the approach for iterative development and testing.

5.6 Discussion

5.6.1 Effectiveness of SSL for Automotive Fault Detection

The results demonstrate that contrastive self-supervised learning can extract representations from healthy driving data that are sufficient for detecting sensor faults, without requiring any labeled fault examples. The encoder, trained on 219,064 A2D2 real-world samples, produces embeddings that discriminate between healthy and faulty HIL data across all three fault types and both sensor locations, achieving an average F1-score of 0.9571 at the 40th percentile threshold.

5.6.2 Domain Alignment: A2D2 to HIL

A core requirement of the pipeline is to align the source domain (real driving data) with the target domain (HIL data used for testing). The results confirm that this alignment is achieved: representations learned from A2D2 data generalize to HIL data, enabling fault detection across domains. Two factors enable this alignment:

1. **Consistent normalization:** The Z-score scaler fitted on A2D2 data (accelerator: $\mu = 6.9852$, $\sigma = 8.7018$; speed: $\mu = 17.2056$, $\sigma = 14.7841$) is applied to HIL data, projecting both domains into a comparable feature space.

2. **Augmentation-induced invariance:** The contrastive learning objective encourages the encoder to capture fundamental signal dynamics (e.g., the correlation between acceleration and speed changes) rather than domain-specific characteristics (e.g., absolute amplitude ranges).

5.6.3 Impact of Threshold Selection

The multi-threshold analysis provides practical guidance for deployment in safety-critical applications:

- For **high-ASIL applications** (e.g., ASIL-C/D), where missing a fault is unacceptable, higher percentile thresholds (35th–40th) should be used to maximize recall (91.5–91.9%). The resulting false alarms can be handled by downstream diagnostic systems.
- For **low-ASIL applications** (e.g., ASIL-A/B), where false alarms are disruptive, lower percentile thresholds (15th–20th) offer fewer false alarms at the cost of lower recall. Only high-confidence detections are reported.
- The (35th–40th) percentile range provides the **best trade-off**, which achieves the F1-score (0.9551 - 0.9571).
- It must be noted that the high Recall achieved at the 40th percentile comes at the cost of a high False Positive Rate: 251 of 316 healthy windows (79.4%) are incorrectly flagged as faulty. In a production vehicle, this would render the system impractical. For real-world deployment, such an operating point would require a secondary verification mechanism—for instance, a temporal persistence filter requiring the fault to be detected in several consecutive windows before triggering an alarm—or integration with a downstream diagnostic system that can tolerate occasional false alarms.

5.6.4 Fault Type Detectability

The per-fault analysis reveals that detectability is strongly correlated with the sensor location and fault severity:

- **Speed sensor faults** are consistently easier to detect than accelerator faults. At the 40th percentile, speed gain achieves 99.7% recall, speed noise 96.5%, and speed stuck 89.9%. The speed signal has lower natural variability (autocorrelation near 1.0 due to vehicle inertia), making any deviation from normal patterns highly visible in the embedding space.
- **Accelerator faults** are harder to detect, with recall ranging from 87.3% (stuck) to 89.1% (noise) at the 40th percentile. The accelerator signal is

inherently more variable (drivers constantly adjust pedal position), and some fault signatures overlap with natural driving variability.

- **Gain faults** on the speed sensor are the most detectable (99.7% at the 40th percentile) because a gain factor applied to speed produces amplitudes that exceed the normal training range.
- **Stuck-at faults** on the accelerator are the least detectable (87.3% at the 40th percentile) because a frozen pedal position can resemble periods of constant throttle during normal driving.

This sensor-dependent and severity-dependent detectability is a *strength* of the approach, as it reflects the physical reality that more severe and more unusual faults are inherently more distinguishable from normal operation.

5.6.5 Limitations

1. **Two-sensor limitation.** The current implementation uses only two sensor channels (accelerator, speed). Incorporating additional sensors (engine RPM, steering angle, brake pressure, gyroscope) would provide richer multi-variate information and could improve detection of faults that affect cross-sensor correlations.
2. **Window-level granularity.** Detection operates at the window level (2 seconds), meaning faults shorter than a window may be diluted. Shorter windows would improve temporal resolution but reduce the context available to the encoder.
3. **No fault localization.** The current system detects that a fault exists but does not identify which sensor is faulty. Ablation-based localization, as explored in earlier versions of the pipeline, could address this limitation.
4. **Threshold dependency.** Recall varies from 49.2% to 91.9% depending on the chosen threshold. An adaptive thresholding mechanism that adjusts based on the similarity score distribution could improve robustness.
5. **ROC-AUC of 0.6314.** While above random (0.5), the ROC-AUC indicates considerable overlap between healthy and faulty similarity distributions. A more powerful encoder architecture, more training data, or additional augmentation strategies could reduce this overlap and improve discrimination.

6 Conclusion and Future Work

6.1 Conclusion

This thesis designed and evaluated a self-supervised contrastive learning pipeline that learns robust representations from real driving data without labels, transfers learned representations to HIL data where faults are known, and enables anomaly detection across domains and scenarios. The framework combines SimCLR-based contrastive pretraining on A2D2 real-world data with cosine similarity-based anomaly detection on HIL fault-injected data. The following conclusions address the research questions posed in Chapter 1.

RQ1: Can a contrastive SSL model detect sensor faults without labeled fault examples?

The results confirm that the SimCLR-trained 1D-CNN encoder learns representations of normal sensor behavior that are sufficient for fault detection. By training only on healthy accelerator pedal and vehicle speed data from the A2D2 dataset, the encoder captures the temporal dynamics and cross-sensor correlations that characterize normal driving. Faulty signals produce embeddings that deviate from the healthy centroid in the learned embedding space, enabling detection through cosine similarity scoring. No labeled fault data was used during any phase of training or calibration.

RQ2: How does threshold selection affect the sensitivity–specificity trade-off?

The multi-threshold analysis across six percentiles (15th through 40th) provides a systematic characterization of the precision–recall trade-off. Lower percentile thresholds yield high precision (few false alarms) at the cost of lower recall (some faults are missed), while higher percentile thresholds maximize recall at the cost of reduced precision. The F1-score identifies the optimal balanced

operating point. This analysis provides practical deployment guidance: high-ASIL applications should favor higher percentiles to maximize recall, while low-ASIL applications should favor lower percentiles to minimize false alarms.

RQ3: Which fault types are most and least detectable?

Speed gain faults are the most detectable (99.7% at the 40th percentile) because a multiplicative gain produces amplitudes that exceed the normal operating range seen during training. This deviation is easily captured in the embedding space, especially for the speed signal which has low natural variability. Noise faults are moderately detectable (96.5% for speed, 89.1% for accelerator) – additive noise alters the signal's high-frequency content but can sometimes be masked by normal driving transients. Accelerator stuck-at faults are the least detectable (87.3% at the 40th percentile) because a frozen pedal position can resemble periods of constant throttle during normal driving (e.g., cruise control, idle). This natural overlap makes detection more challenging.

6.1.1 Key Contributions

The thesis makes the following contributions:

1. A self-supervised contrastive learning framework for automotive sensor fault detection that requires zero labeled fault data, addressing the primary limitation of existing supervised methods.

2. Demonstration of cross-domain transfer from real-world A2D2 driving data to HIL simulation data, validating the practical deployment scenario where a model trained on production vehicles is applied to laboratory test benches.

3. A systematic multi-threshold evaluation (15th–40th percentile) with comprehensive metrics—precision, recall, F1-score, accuracy, confusion matrices, ROC curves, and computing costs—providing a complete characterization of detection performance and practical deployment trade-offs.

4. Detailed documentation of the entire pipeline, from data loading and preprocessing through SimCLR training to anomaly detection and evaluation, at a level of detail that enables reproduction by other researchers.

6.2 Future Work

Several directions are identified for extending this research:

1. **Multi-sensor extension.** The current two-sensor system should be extended to include additional automotive sensors—engine RPM, steering angle, brake pressure, and inertial measurement unit readings. A larger sensor set would provide richer multi-variate patterns and could enable sensor-level fault localization through ablation analysis, where each sensor is individually masked and the resulting change in similarity score indicates its contribution to the detected anomaly.

2. **Alternative SSL frameworks.** Comparing SimCLR with other contrastive methods such as TS2Vec (Yue et al., 2022), MoCo (He et al., 2020), and BYOL (Grill et al., 2020) would establish which framework produces the most discriminative representations for automotive time-series data. TS2Vec is particularly promising because it is designed specifically for time-series and captures multi-scale temporal patterns.

3. **Fault severity estimation.** Beyond binary detection (healthy/faulty), the magnitude of the similarity deviation from the healthy centroid could be calibrated to estimate fault severity. This would enable prioritized alerting, where severe faults trigger immediate responses while minor anomalies are logged for later review.

4. **Adaptive thresholding.** A fixed threshold may not be optimal across all driving conditions. Developing an adaptive threshold that adjusts based on the current driving context (e.g., highway vs. urban driving) could reduce false alarms during dynamic maneuvers while maintaining sensitivity during steady-state operation.

5. **Real vehicle deployment.** The ultimate validation of the approach requires deployment on real vehicle sensor data from production test drives. This would test the framework under conditions of genuine sensor degradation, environmental interference, and the full diversity of real-world driving.

6. **Enriched datasets.** The current work used only healthy data for training and purely faulty files for testing. Future work should include datasets that contain a mixture of healthy and faulty windows, more closely resembling real-world conditions. Additionally, expanding the

healthy training data with more diverse driving scenarios could improve representation robustness.

7. **Fault classification and localization.** While this thesis focused on binary anomaly detection, extending the framework to classify fault types (gain, noise, stuck-at) and localize the faulty sensor would provide more actionable diagnostic information. This could be achieved by training a classifier on the learned embeddings or by using ablation techniques to identify which sensor contributes most to the anomaly score.

Bibliography

Abboush, Mohammad et al. (2022). "Intelligent Fault Detection and Classification Based on Hybrid Deep Learning Methods for Hardware-in-the-Loop Test of Automotive Software Systems". In: *Sensors* 22.11, p. 4066.

Audi AG (2020). *A2D2: Audi Autonomous Driving Dataset*. `https://www.a2d2.audi/`.

Chen, Ting et al. (2020). "A Simple Framework for Contrastive Learning of Visual Representations". In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*. PMLR, pp. 1597–1607.

Chen, Xiaohan et al. (2023). "Transfer Learning for Bearing Fault Diagnosis: A Systematic Review since 2016". In: *IEEE Transactions on Instrumentation and Measurement* 72, pp. 1–22.

Chiaroni, Florent et al. (2021). "Self-Supervised Learning for Autonomous Vehicles Perception: A Conciliation Between Analytical and Learning Methods". In: *IEEE Signal Processing Magazine* 38.1, pp. 31–41.

Ding, Yifei et al. (2022). "Self-Supervised Pretraining via Contrast Learning for Intelligent Incipient Fault Detection of Bearings". In: *Reliability Engineering & System Safety* 218, p. 108126.

Eldele, Emadeldeen et al. (2021). "Time-Series Representation Learning via Temporal and Contextual Contrasting". In: *Proceedings of the 30th International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 2352–2359.

Ghannoum, Ehab (2025). "Explainable AI for Deep Learning-Based Fault Detection and Diagnosis of Automotive Software Systems During the Real-Time Validation Process". MA thesis. TU Clausthal.

Grill, Jean-Bastien et al. (2020). "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning". In: *Advances in Neural Information Processing Systems*. Vol. 33, pp. 21271–21284.

Gutmann, Michael and Aapo Hyvärinen (2010). "Noise-Contrastive Estimation: A New Estimation Principle for Unnormalized Statistical Models". In: *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics*, pp. 297–304.

He, Kaiming et al. (2020). "Momentum Contrast for Unsupervised Visual Representation Learning". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 9729–9738.

Hojjati, Hadi, Thi Kieu Khanh Ho, and Narges Armanfard (2024). "Self-Supervised Anomaly Detection: A Survey and Outlook". In: *arXiv preprint arXiv:2205.05173*.

Ioffe, Sergey and Christian Szegedy (2015). "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift". In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pp. 448–456.

ISO 26262: Road Vehicles – Functional Safety (2018). International Organization for Standardization.

Jaiswal, Ashish et al. (2021). "A Survey on Contrastive Self-Supervised Learning". In: *Technologies* 9.1, p. 2.

Kingma, Diederik P and Jimmy Ba (2015). "Adam: A Method for Stochastic Optimization". In: *arXiv preprint arXiv:1412.6980*.

LeCun, Yann, Yoshua Bengio, and Geoffrey Hinton (2015). "Deep Learning". In: *Nature* 521.7553, pp. 436–444.

Matos, Francisco et al. (2024). "A Survey on Sensor Failures in Autonomous Vehicles: Challenges and Solutions". In: *Sensors* 24.16, p. 5108.

Oord, Aäron van den, Yazhe Li, and Oriol Vinyals (2018). "Representation Learning with Contrastive Predictive Coding". In: *Advances in Neural Information Processing Systems*.

Pang, Guansong et al. (2021). "Deep Learning for Anomaly Detection: A Review". In: *ACM Computing Surveys* 54.2, pp. 1–38.

Ruff, Lukas et al. (2021). "A Unifying Review of Deep and Shallow Anomaly Detection". In: *Proceedings of the IEEE* 109.5, pp. 756–795.

Waeltermann, Peter (2016). *Hardware-in-the-Loop: The Technology for Testing Electronic Controls in Vehicle Engineering*. `https://www.dspace.com/en/pub/home/news/hil-simulation-paper.cfm`.

Yue, Zhihan et al. (2022). "TS2Vec: Towards Universal Representation of Time Series". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 36. 8, pp. 8980–8987.

Zhang, Tianci et al. (2022). "Intelligent Fault Diagnosis of Machines with Small and Imbalanced Data: A State-of-the-Art Review and Possible Extensions". In: *ISA Transactions* 119, pp. 152–171.

Zhang, Xiang et al. (2022). "Self-Supervised Contrastive Pre-Training for Time Series via Time-Frequency Consistency". In: *Advances in Neural Information Processing Systems* 35, pp. 3988–4003.